Masters Thesis

On the Effectiveness of Deep Learning Techniques for Malware Detection and their Resiliance to Obfuscation

A thesis submitted in fulfilment of the requirements for the degree of Master of Research in Advanced Artificial Intelligence

in the

School of Engineering and Informatics
University of Sussex

Author:
Henry Williams

Supervisor:
Jeff Mitchell

Abstract

Malicious software is an extremely pervasive threat faced by both individuals and organisations, that has the potential to cause significant damage to computer systems. We explore the usage of deep learning techniques for automated identification of malware, and evaluate how resiliant these approaches are to obfuscation techniques used to bypass existing malware detection services. We present a natural langauge processing based approach, using the BERT architecture for malicious opcode sequence classification, and an image-based approach using the EfficientNet architecture. We find that these static malware detection approaches consider obfuscation as highly indicative of malware, leading to misclassification of obfuscated benign executables.

August 2025

Statement Of Originality

This report is submitted as part requirement for the degree of Masters of Research in Advanced Artificial Intelligence at the University of Sussex. It is the product of my own labour, except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby give permission for a copy of this report to be loaned out to students in future years

Signed,

H.Williams

Henry Williams
Candidate No: 233561

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Malware is one of the most significant threats faced by users in the current digital landscape. The increasingly sophisticated nature of these programs, in terms of both their capabilities and their stealth, mean that this threat is only increasing as we become evermore reliant on digital infrastructure in our daily lives.

Therefore, detecting malware before it can cause damage is of high priority to security professionals. However, following the introduction of anti-virus software, malware developers of have used various evasion techniques to avoid detection. This has led to a cat-and-mouse like game, where cybercriminals adopt new forms of evasion techniques, and security researchers developing new techniques to handle these new forms of obufscation. This makes conventional approaches to malware detection challenging, due to its constantly evolving nature.

Machine learning however offers a potential means of dealing with this by finding ways to detect malware in spite of evasion techniques. In this investigation, we use various approaches from a number of machine learning domains, such as linear modelling, natural langauge processing, and image processing to investigate both their performance at detecting malware targeting the windows operating system, and their resiliance to obfuscation.

We evaluate both a RoBERTa-based language model trained on sequences of assembly instruction, and Convolutional Neural Network using image representations of programs to identify malware. These are both considered static approaches where feature are extracted without executing the program. Dynamic methods on the other hand, observe the behavior of a program, and are typically more capable than static methods. However, they often incur a significant overhead in their complexity, as an environment where a potentially malicious program can be safely executed is required. This is typically a remote server hosting a virtual machine that isolates the execution, preventing it from causing harm. However, to communicate with this safe execution environment, an internet connection is required. In comparison, static approaches are able to operate in environments where an internet connection is not guaranteed. Therefore, in light of their disadvantages, static methods are still relevant in domain.

Due to the highly pervasive nature of obfuscation in modern malware, we additionally theorise that static approaches heavily rely on the presence of obfuscation as an indicator for malware. However, obfuscation is not exclusive to malicious programs, as legitimate software developers may use it to protect their intellectual property from reverse engineering. Therefore, to better understand how static approaches identify malware, we conduct an additional experiment that aims to evaluate how our models distinguish between benign, and malicious programs.

The research questions we seek to answer in this work are as follows:

RQ1: *Are modern natural language processing techniques, such as language models and pretraining effective at identifying malware?*

RQ2: *Does pretraining improve the accuracy and reliability of malware detection models based on techniques from natural language processing?*

RQ3: *To what extent are static malware detection models resiliant to obfuscation?*

1.1 Professional Considerations

This research is conducted in strict accordance with the BCS Code of Conduct (British Computing Society, 2022), ensuring adherence to the ethical, legal, and professional standards expected of computing professionals. The work is structured to protect the public interest, maintain professional competence and integrity, uphold obligations to the relevant authority, and safeguard the reputation of the profession.

Public Interest

We conduct our research with regard to the safety, rights, and welfare of the public to ensure our work is in their interest. Samples of malicious programs will be handled in a secure environment, with safeguards to protect against execution on both our own hardware, and that of others to avoid accidental infection. Furthermore, proprietary programs subject to copyright law will not be distributed to ensure the intellectual property rights of third parties are respected. Our research will be conducted without discrimination on the basis of ethnicity, sex, religion, age or other protected characteristsics, and we distribute all materials used to conduct this investigation to support transparency, reproducibility, and to benefit the wider research community.

To ensure the malware samples used in this investigation do not accidentally infect any of our own hardware, or the systems of others, we prevent these programs from being accidentally executed using the `chmod` program. This is a tool in UNIX based systems that modifies the permissions associate with a file, and can prevent executable files from being run on the system.

Professional Competence and Integrity

We conduct our research within the verified level of competence of the authors, and avoid misrepresenting our skills and expertise. The project itself contributes to the continued professional development of our skills in Artificial Intelligence, and Cybersecurity. We employ established best practices in both fields where appropriate, and comply with all relevant UK legislation, including data protection, intellectual property, and cyber-security. Diverse perspectives are actively sought, and the authors welcome constructive criticism to ensure the quality and robustness of the work. No element of this research will cause harm to the property, reputation, or the livelihood of others through false or negligent actions.

Duty to the Relevant Authority

To ensure the duty of the authors to relevant authorties, we conduct this research in compliance with the policies and procedures of the University of Sussex, and any other governing bodies. Any potential conflicts of interest are avoided, and disclosed in cases where they are unavoidable. We accept any professional responsibilities for our work, and that of others involved with this project. Results and findings will be reported accurately, and truthfully, without distortion, omission, or misrepresentation.

Duty to the Profession

The authors acknowledge their responbility to uphold the credibility and reputation of the computing profession, and we pledge to take no actions that will take it into disrepute.

Chapter 2

Background

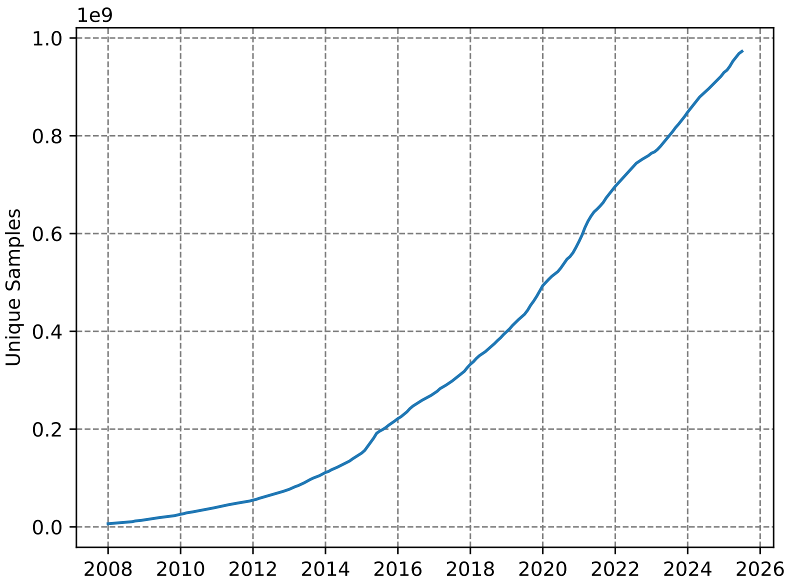


Figure 2.1: Observed growth in the number of unique instances of malware over time. Data taken from (AV-TEST - The Independent IT-Security Institute, 2025).

Malicious software, also known as malware, is defined as any software that has an adverse impact on the typical operation of a system. This could include the extraction of sensitive information, hijacking of system resources, and modification of information. They are considered one of the most pervasive threats faced by both individuals and organisations in the current digital threat landscape (European Union Agency for Cybersecurity, 2024; Alenezi et al., 2020).

This threat can been seen in infamous cyberattacks such as the 2022 Viasat hack (Quiquet, 2023), and the 2017 WannaCry incident (National Audit Office (NAO), 2017; Akbanov et al., 2019). In both of these cases, sophisticated malware were used to disrupt critical infrastructure, including communication systems across Europe, and the services used by the National Health Service.

In recent years, the sophistication (Alenezi et al., 2020; Baezner et al., 2017) and prevalance (Figure 2.1) of malware have increased. Therefore, there has been a great deal of interest into advanced techniques for the reliable detection of malware (M. et al., 2023; Maniriho et al., 2024; Aboaoja et al., 2022) within the information security field. In this chapter, we provide a background on characteristics commonly associated with malware, and discuss various techniques proposed by security researchers to better handle this threat.

2.1 Malware

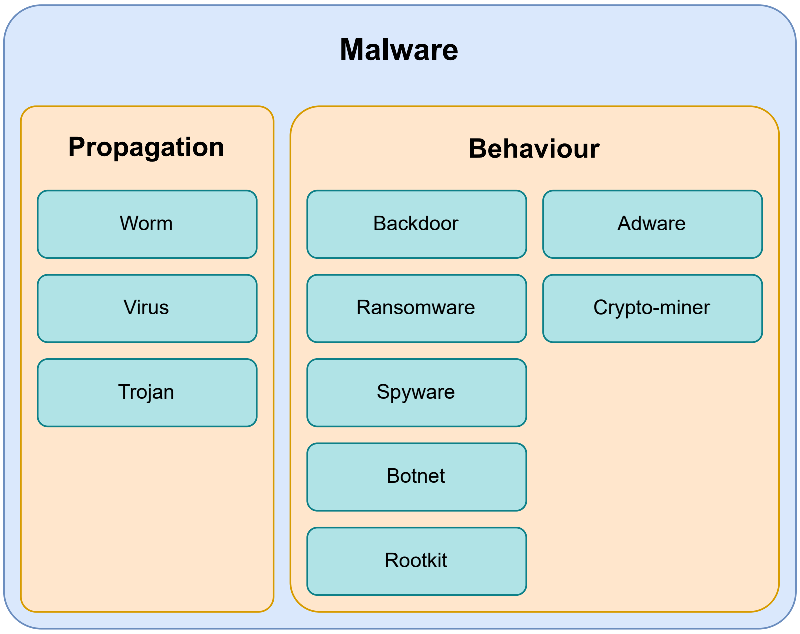


Figure 2.2: Categories of Malware, including both the means of propagation and behaviour on an infected host.

As outlined in the previous section, malware is a form of software that has a detrimental impact on the expected operation of an infected system. However both this and more formal definitions, such as the one presented in (Kramer et al., 2010), fail to define what is meant by harmful. Therefore, we define "harmful" behaviour as any actions in violation of the Confidentiality-Integrity-Availability (CIA) triad (Cochran, 2024). This is a principle that aims to guide the development of security policies to ensure the security of digital systems, and the information stored on them. It states that a system must be inaccessible to unauthorised users (Confidential), accessible to authorised users (Available), and the protected against the modification or deletion of the information it stores (Integrity). Malware provides a comprehensive means of violating this triad of security principles, and it is for this reason we use it in our definition.

Malware is categorised based on its behaviour, and how it spreads between hosts. We present the following taxonomy based on the work of Maniriho et al., 2024 shown in Figure 2.2 that outline the different kinds of malicious behaviors, and propagation techniques. Note that categories within this taxonomy are not mutually exclusive, as malware often belong to multiple categories at once.

- **Worm** — Malicious software that automatically propagates throughout a network.
- **Virus** — Programs that modify other programs to replicate themselves throughout an infected host.
- **Trojans** — Malware that disguises itself as a legitimate program to infect a host.
- **Backdoor** — Malware that provides covert remote access to infected systems.
- **Ransomware** — Malware that denies access to systems or information until a fee is paid.
- **Spyware** — Malware that spy on the user without their knowledge, often extracting information such as credentials, keyboard logs, and live screenshots of the user's activity.

- **Botnet** — Malware that integrate infected systems into a coordinated network of agents.
- **Rootkit** — Malware providing privileged access to an infected system.
- **Adware** — Malware that display advertisements on the host machine.
- **Crypto-miner** — Malware that hijack the user's hardware to mine cryptocurrencies.

2.1.1 EVASION TECHNIQUES

The earliest forms of malware detection used a collection of unique signatures associated with malicious programs(A. Saeed et al., 2013). To counter this, the developers of malicious software began using evasion techniques that aim to decieve these systems to maximise the amount of potential damage their software is able to do before it is identified and removed. These include obfuscation, and environmentally aware execution. In the following sections, we breifly discuss these techniques, and how they are effective in bypassing detection.

Obfuscation

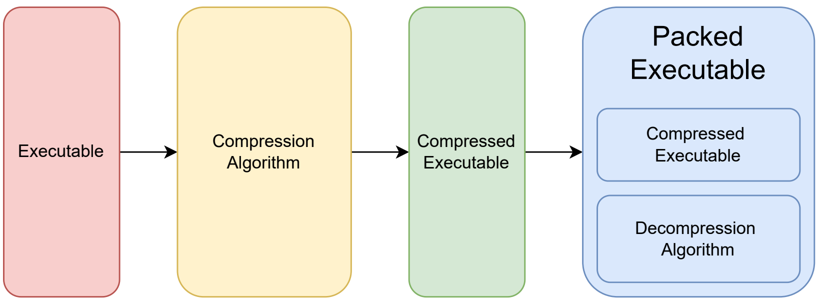


Figure 2.3: How binary packing can be used to reduce the size of an executable program, and obscure its behaviour.

Obfuscation refers to the act of making something more challenging to understand to an external observer. In the context of malware, it includes techniques which alter the syntactic structure of a program, yet maintain the original behavioural characteristics.

Register re-assignment is one of the earliest forms of obfuscation in which the registers used by the program are randomly reallocated (Balakrishnan et al., 2005). Registers are small areas of memory within the CPU that store information used during the program's execution. By changing which registers are used to store this information, the physical structure of the program is changed without impacting its behavior. This defeats simple malware detection solutions that use the unique cryptographic hashes of a program as its signatures, as a unique representation of a program will give a distinct hashes that has not been seen before, and therefore not identified as malicious.

We can also modify the structure of a program by inserting instructions, or functions which either have no impact on the program state, or are never executed. This is known as dead-code insertion, and is a relatively common form of obfuscation, as it can provide vastly different representations of the same program. However, so-called "dead-code" can easily be identified and removed, by either human experts, or a call flow graph.

Code transposition modifies the order of code the program (Christodorescu et al., 2004) to obscure its signatures by moving small units of code known as basic blocks. To ensure the program behaves as expected, all references to the relocated sections are updated to its new location. This again evades detection by these early signature based methods, as many unique representations of the same program can be generated.

Despite the ability of these techniques to trick early anti-malware approaches, more sophisticated approaches are more resiliant to these forms of obfuscation. These approaches use an intermediate representation of the program, that normalise the structure of a program resulting in a representation that is indentical to an un-obfuscated one. To decieve these advanced approches, more sophisticated forms of obfuscation are needed. One such example of this is known as binary packing, where the malicious portion of code is encoded and stored as data within another program that decoded and executes the payload (Figure 2.3). Encoding can take many forms, such as encryption or compression, but they typically make it challenging to retrieve the decoded payload without running the program. However, the portion of the packed binary responsible for decoding and execution will often remain constant, meaning it can be used for identification. Some malware developers employ the aforementioned obfuscation techniques to obscure the decoder and avoid detection which are known as Oligomorphic and Polymorphic malware (You et al., 2010).

Environmental Awareness

The techniques demonstrate the challenges of relying on the static strucuture of a program to determine its nature, and it is for this reason that dynamic features are considered to be more robust at identifying malicious programs. This is because the behaviour of a program is invariant to these forms of obfuscation. Dynamic features are observed at execution time and include sequences of system API calls, modifications to the filesystem and system configuration, and internet traffic. However, this requires running a potentially malicious program, which could have harmful effects on the host system. Therefore, the execution of a potentially malicious program will occur within a sandbox, an environment where harmful actions can occur with no implications on the security of the system. This sandbox is typically a virtual machine that is representative of the software's intended target.

To decieve these approaches, the malware developers will have their code check if it is running within a virtual machine before it executes. Common inidicators of a virutal machine used by malicious programs include system hardware characteristics (CPU core count, RAM size, etc), the presence of shared libraries used to communicate between the sandbox and its host, and the number of processes running on the system to name a few. If the program determines that it is likely to be running within a sandbox, it will refuse to execute the malicious portion of the code, and will exit. This prevents the observer from gaining any information about its behavioural characteristics, preventing the creation of dynamic malicious signatures.

2.2 Related Work

Malware detection is a challenging problem, and many approaches have been proposed since the first forms of malicious software were seen in the 1980s. The earliest form of malware detection used a database containing the unique signatures of malicious programs. While similar signature based approaches are still in use today, there is much interest in advanced methods, that are robust to evasion techniques, and that able to identify novel forms of malware (M. et al., 2023; Merabet et al., 2019; Aboaoja et al., 2022; Idika, Mathur, 2007). These advanced techniques employ a wide variety of technologies, such as multiple sequence alignment of API call sequences (Ki et al., 2015), machine learning (Kamboj et al., 2023), and deep learning.

These techniques typically fall into one of two categories, based on how the features used to classify programs are extracted, which are as follows.

- **Static** — Features extracted from the structure of a program
- **Dynamic** — Features extracted during the execution of the program

Static approaches use features extracted from the contents of an executable as opposed to its behavior during execution. Common features used to identify malware include greyscale images of the executable (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023), printable strings (Tian et al., 2009; Singh et al., 2020; Schultz et al., 2001), and sequences of instructions (Kakisim et al., 2022; Wang et al., 2021; Darem et al., 2021).

In Schultz et al., 2001, the authors use static features such as printable ASCII strings to train various machine learning classifiers to detect malware. In their experiments, they found that the Naive Bayes algorithm was best suited to the problem, achieving 97% classification accuracy. This is one of the first papers that introduce the usage of machine learning in this domain, and the high accuracy of their experiment shows its potential over conventional signature or heuristic methods. However, the dataset used in this work was small, and featured an unbalanced class distribution of 3265 malicious programs, and only 1001 benign which would likely have an adverse effect on the generalisability of their proposed models. Furthermore, malware often obfuscate strings that are decoded during execution, which would be sufficient to defeat their proposed approach.

Much of the literature relating to static malware detection use convolutional neural networks (CNN) to distinguish between image-based representations of malicious and benign executables (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023). In Aslan et al., 2021, the authors represent an executable as a greyscale image, and train a CNN to identify malware. While their proposed approach is able to effectively distinguish between the different types of programs, CNNs require constant input size. Therefore, resizing of the program's image representation is necessary. Malicious programs are often smaller than benign programs, meaning that in datasets where there is significant variation in size between classes, compression artifacts may be more prevalant in once class, leading to the model developing an insufficient solution. In Sharma et al., 2019, the authors also use a CNN to detect malware. Instead of using a 2-dimensional CNN however, the authors opt to use a 1-dimensional variant. This led to better performance than other CNN based approaches, as the vertical spatial dependence, which is largely irrelevant in malware detection, is removed. However, it still has the same requirement as the previously discussed approach, which could lead to a partial solution.

Instruction sequences have also been explored as static features in malware detection. For example, in (Attaluri et al., 2009; Runwal et al., 2012), the authors utilise hidden markov models and achieve promising results. As these models consider what actions are being performed by the host system without having to execute the program, they offer a solution that we believe to be more resilient to obfuscation than other static approaches.

Extending upon the idea of utilising instruction sequences for malware detection, (Demırcı et al., 2022) outline an approach in which they propose the use of stacked bidirectional Long-Short Term Memory (BiLSTM) and pre-trained language models including BERT (Devlin et al., 2019), and GPT-2 (**radfordLanguageModelsArea2019**). In their investigation, the BiLSTM model was able to achieve 98% classification accuracy whereas the pre-trained transformer models achieved only 77%. Transformer models have been shown to be a powerful technique for sequence classification problems, and as such, this result is surprising. We believe this to be a result of the corpora used to pretrain the transformer models used in their investigation, which are comprised of mostly english texts. The understanding of these models would thus be limited to languages similar to english, and may struggle to understand the intricacies of assembly language, impacting their ability to reliably classify malware. The Bi-LSTM model on the other hand, is trained from a random initialisation, and therefore is able to develop a better understanding of malicious characteristics in opcode sequences.

Previous research has also established the use of pre-trained language models in malware detection using the metadata of an executable. In Rahali et al., 2021, the authors fine-tune BERT on text gathered from the manifest file found in android applications. This file contains information pertaining to the activities, background tasks, event listeners, and permissions granted to an application. Their model was able to achieve 97% binary classification accuracy, in comparison to (Demırcı et al.,

2022) which was only able to achieve 77% with the same model using opcode sequences. We believe this discrepancy to be a result of the language found in these manifest files being more closely related to the pre-training corpora of BERT. This would enable their model to better understand the information found in the manifest, and its use in distinguishing between benign and malicious executables. Their approach however, is limited to environments where a file containing metadata are required, such as mobile operating systems, making it unsuitable for desktop malware detection, where such metadata files are not always present.

The performance of static malware detection approaches vary, depending on the detection method used, the availability of data, and other factors, but it is well established that they are particularly vulnerable to obfuscation. In Bacci et al., 2018, the authors investigate the impact of obfuscation on the performance of static and dynamic malware detectors on android devices. They found that while dynamic methods are not significantly impacted by the presence of obfuscation, static techniques are more affected. However, by training static methods on obfuscated programs, it is possible to mitigate this problem to some extent.

Furthermore, dynamic methods generally tend to be more capable than static methods (Damodaran et al., 2017). This is somewhat to be expected, as behaviour is more indicative of malicious actions, especially when obfuscation is used to hide a program's intent. However, dynamic methods require a sandbox to execute a program for analysis, without causing damage to a host. This typically takes the form of a remote, virtual machine as running such a sandbox locally would have a significant computational cost. As a result of this, dynamic methods require both significant compute resources, and an internet connection, which is not guaranteed. It is for this reason that static methods, that can identify potentially malicious programs, without an internet connection is needed for enhanced protection from this threat.

In addition to the high resource requirements associated with dynamic feature extraction, sophisticated evasion techniques such as environmentally aware execution provide malware developers with the capabilities to obscure the dynamic features by preventing execution within virtual machines used to extract dynamic features. Furthermore, novel obfuscation techniques, such as the one proposed by Li et al., 2023, is able to obscure which system API calls are made by a program. This is a widely used dynamic feature for detecting malware, and the proposed technique enables the malware to call these system APIs without triggering system hooks used to track which methods are called during a program's execution. Therefore, despite offering better performance than static methods, dynamic methods are also vulnerable to evasion techniques.

2.3 Datasets

Research into malware detection depends on the availability of large scale, high quality datasets containing both malicious and benign software. However, very few of these datasets exist, and those that do often provide features derived from the programs as opposed to the executables themselves. For example, the EMBER dataset (Anderson et al., 2018) contains over a million samples of static features derived from each executable, however the binaries themselves are not distributed. Other datasets, such as BODMAS (Yang et al., 2021), despite offering fewer samples than EMBER, provide access to the executables. However, due to copyright limitations, only malicious binaries are provided.

When curating these datasets, malicious samples are easy to download en-masse through websites such as VirusShare[1] and VirusExchange[2]. These websites store large collections of malware seen in-the-wild by security professionals. However, similar large scale collections of benign software are less available, likely due to copyright protections. We have seen a number of approaches for curation of benign software in the literature. For example, by downloading android applications

[1] `https://virusshare.com`

[2] `https://virus.exchange/`

from the Google play store (Karbab et al., 2018), scraping free software websites (Li et al., 2022), and extracting system executables. These approaches however are somewhat limited by the number of programs available on both free software websites, and the system itself. Furthermore, these executables are not guaranteed to be free of malware. Another promising approach for collecting benign windows software is the downloading of repositories used by windows package managers such as chocolatey and Winget as they provide many packages that have been verified as being free of malware. However, they often distribute installers rather than the executables themselves, meaning each package has to be installed before it can be used in the dataset.

2.4 Future Research Directions

Our analysis of the literature relating to this field, has found that there are a number of promising areas of investigation. Static approaches, while less robust to structural obfuscation, are less resource intensive and are able to run on a users machine without an internet connection. Dynamic features on the other hand, offer better performance than its static counterparts, yet are still vulnerable to advanced evasion techniques that may render them less effective than static. Furthermore, the requirements associated with dynamic feature extraction are a significant barrier to their deployment, and demonstrate the need for efficient and effective static methods for detecting malware.

Furthermore, we identify a gap in the literature relating to the use of modern natural language processing techniques such as pretraining transformer models for malware detection on textual representations of executables. While these models have been used in this context, they use the natural langauge features of these programs, such as the ascii strings, and program metadata. We will investigate the use of these techniques when trained to develop an understanding of the program itself to see if these modern NLP techniques are suited to reasoning about executables with the end goal of identifying malware.

In addition to this, we will also investigate the resiliance of static approaches to obfuscation, as it seems that the it is present in most malware seen in the wild. This may cause static malware detection models to recognise harmful software based on the presence of obfuscation. This will develop a better understanding of the issues associated with static malware detection which may lead to further advancements in the field.

Chapter 3

Methodology

In this investigation, we evaluate the effectiveness of a custom RoBERTa model in static malware detection. The model is trained to distinguish between malicious and benign opcode sequences using modern natural langauge processing techniques such as pretraining, which are then used to identify malicious programs. We compare this model against both linear models, such as decision trees, and other static malware detection methods seen in the literature, such as Convolutional Neural Networks. Finally, we evaluate the resiliance of the opcode sequence classificatio, and the CNN image classification models against obfuscated malware to assess their inner workings.

3.1 Dataset

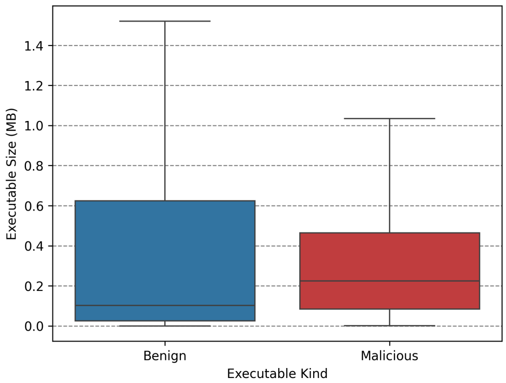
Our dataset consists of 11115 windows PE32 files (See appendix A.6), with a class distribution shown in fig. 3.1c. Malicious files are gathered from VirusShare[1], which contains samples from real-world cybersecurity incidents, providing approximately 90 million samples of which nearly 6000 were randomly selected. The benign executables were collected from the personal devices of the authors, and were scanned using Microsoft defender to ensure no mislabelled samples were present. This includes various software packages, games, developer tools, system programs, and other miscellaneous executables to ensure the dataset is representative of the kinds of benign programs commonly seen on the machines of potential end users.

We opt to investigate windows malware detection due to the operating systems widespread usage, and therefore high availability of malware targeting it. Malware that target other operating systems do exist, but there are significantly fewer samples, meaning the availability of data is reduced.

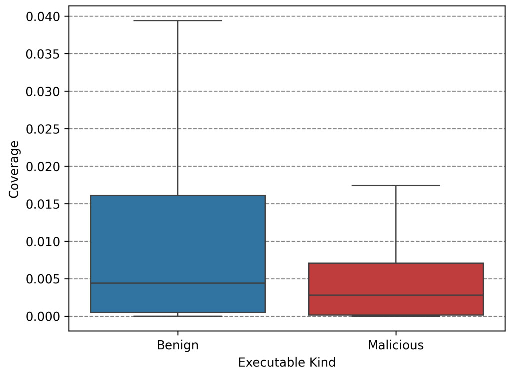
The dataset demonstrates wide variation in characteristics between classes. For example, we find that malicious programs are often much smaller than benign executables, as shown in fig. 3.1a. We believe this to be because benign programs typically provide more functionality, such as graphical user interfaces, telemetry, and compatability, which increase the size of the program. Malicious software on the other hand, are developed with a single goal in mind, reducing its size.

We hypothesize that both our dataset, and other datasets containing malicious software will demonstrate variation in the prevalance of obfuscation between malicious and bengin executable classes. To investigate this, we perform static analysis of the samples in our dataset using Rizin (Rizin Organization, 2025) to find the code analysis coverage. This metric gives the ratio of code analysed by the tool, to the total amount of code in an executable. In obfuscated programs, this result is typically much lower than non-obfuscated executables as it is unable to perform its analysis due to the non-standard structure. The results of this analysis are presented in Figure 3.1b, which supports our hypothesis, and indicates malicious samples are more frequently obfuscated than benign samples.

[1] `https://virusshare.com/about`



(a) Distribution of executable size across classes.



(b) Distribution of code analysis coverage across classes.

(c) Distribution of classes with our dataset.

Class	Count
Malicious	5792
Benign	5323
Total	11115

Figure 3.1: Description of our dataset, including the size of programs (a), the extent of obfuscation (b), and the total number of samples in each class (c).

The dataset is then split to produce a training set, and evaluation set at a ratio of 80% to 20% of samples respectively. To ensure consistent class distribution between these sets, samples are shuffled prior to splitting.

3.1.1 OBFUSCATION EXPERIMENT DATASET

Table 3.1: Distribution of classes with our obfuscation experiment dataset.

Class	Count
Malicious	60
Benign	61
Total	121

To investigate the impact of obfuscation on malware detection, we also curate a secondary dataset, consisting of unobfuscated executables. We use the work presented in (Rokon et al., 2020) to identify github repositories containing malicious source code. The authors provide a list of URLs to these repositories, from which we select windows programs that provide an executable and with no mention of obfuscation in its description or codebase. Benign samples consist of Windows system binaries not present in our original dataset. We identify a collection of approximately 60 windows XP system binaries which are free of obfuscation for this secondary dataset. This gives a selection of 121 programs, each of which are free of obfuscation for use in our later experiment into the resiliance of machine learning algorithms to obfuscation.

3.2 Metrics

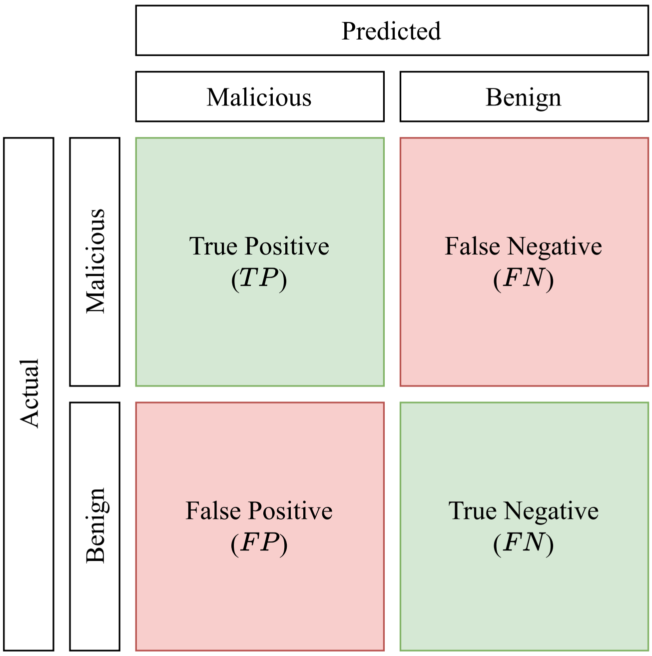


Figure 3.2: Confusion Matrix describing the kinds of classification results in malware detection.

In our investigation, malware detection is presented as a binary classification problem, where our models aim to distinguish between malicious and benign software. To quantify the performance of the various approaches tested in this investigation, we find the accuracy, f1-score, precision, and recall (See eq. (3.1)). These metrics are based on the different kinds of results a binary classifier can produce, shown in fig. 3.2.

Accuracy is the ratio of correct classifications to the number of samples tested, where 1 would be a perfect classifier for a given set of samples. However, the value of this metric can be misleading in unbalanced datasets.

Precision and recall both operate on the class-level, therefore to reduce these metrics to a single number, we typically take the average of these values for each class in the dataset, known as macro-averaging. Precision is definied as the ratio between the number of correctly identified samples, to the total number of samples predicted as belonging to a given class. Recall on the other hand is the number of correct predictions, to the number of samples in given class. F1 score, acts as an aggregation of these values, and is given by the harmonic mean.

$$\begin{aligned}
\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\
\text{Precision} &= \frac{TP}{TP + FP} \\
\text{Recall} &= \frac{TP}{TP + FN} \\
\text{F1-Score} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}
\end{aligned} \tag{3.1}$$

3.3 Baseline

Similarity	Malicious	Benign
Malicious	0.1428	0.0931
Benign	0.0931	0.1217

(a) Cross-class external function similarity.

Similarity	Malicious	Benign
Malicious	0.2249	0.1606
Benign	0.1606	0.1643

(b) Cross-class DLL similarity.

Figure 3.3: Cosine similarity between classes based on external functions and DLLs.

We firstly establish a baseline using conventional machine learning algorithms on the lookup table associated with each program. This lookup table tells the program where external references, such as libraries and functions exist in memory. These external references offer functionality that would otherwise be inaccessible to the program. For example, to interact with the file system, the lookup table will include a reference to `KERNEL32.dll`, which provides the address of the functions needed to create, read, and write to the file system. We can extract all functions and libraries used by an executable with other static analysis tools such as radare2 (Radare Organization, 2025). References to functions that occur once across the dataset are removed as approximately half of all functions are seen exactly once across the entire dataset. This gives two binary feature vectors for each sample in the dataset, telling the classifiers which external functions, and libraries are used by the program. We test 3 machine learning algorithms to establish this baseline, which are support vector classifiers, logistic regression, and decision trees.

We also use the feature vectors to quantify the similarity of programs in our dataset with the pairwise cosine similarity function (Equation (3.2)). We then find the average similarity between classes which are presented in Figure 3.3. From this, we can assume that functions and libraries used by the malicious programs are more similar than the ones used by benign programs. Furthermore, functions used by malicious programs are vastly different to functions used by benign programs.

$$S(\mathbf{A}, \mathbf{B}) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{3.2}$$

3.4 RoBERTa for Opcode Sequence Classification

```
ret lea mov add add lea mov add add nop
```

Figure 3.4: An example sequence of opcodes

In recent years, transformer-based architectures for natural language processing have been shown to be highly effective across a wide variety of language based tasks. A significant advancement within the field was the development of the Bidirectional Encoded Representation from Transformers model, known as BERT (Devlin et al., 2019). This model uses an encoder-only architecture, based on the multi-head self-attention mechanism (Vaswani et al., 2023) in conjunction with a pretraining stage to develop a comprehensive understanding of language. Pretraining consists of two tasks, masked language modelling where the model predicts hidden tokens based on their surrounding context, and next sentence prediction. The pretrained model can then be fine-tuned on a downstream problem such as sequence classification, question answering, and general language understanding. At the time of publication, it achieved state-of-the-art performance in the General Language Understanding Evaluation (GLUE), Stanford Question Answering Dataset (SQuAD), and Situations With Adversarial Generations (SWAG) benchmarks and has been used extensively throughout the natural language processing field. Fine-tuning of the pretrained model allows researchers to solve tasks with fewer training samples, short training periods, and less intensive compute requirements. RoBERTa extends upon the original BERT model by expanding the scale of the model, removing the next sentence prediction task from its pretraining stage, optimising the hyperparameters, and training on a larger corpus.

We develop a custom RoBERTa model, trained on opcode sequences for malware detection. Opcodes refer to the operation to be performed by the CPU, such as `add`, `cmp`, `mov`, written in a human readable form. To generate these sequences, we firstly use radare2 to extract the disassembly of a program (See algorithm 1), and remove the operands of each instruction (algorithm 2) to reduce the vocabulary size and therefore the computational resources needed to train the model. This gives a sequence of opcodes representing each program within our dataset, as shown in fig. 3.4. The sequences are then tokenized at the word level where each opcode is assigned a single token.

We train four of these models which each take a tokenized opcode sequence of some fixed length. The first model is trained in masked language modelling, which is then used to train a sequence classification model, refered to as the “pretrained” model throughout this report. We also train another pretrained sequence classification model which has had the weights in its attention layers frozen, refered to as the “frozen” model to determine if the representations produced by the attention layers are sufficient for distinguishing between malicious and benign opcode sequences. Finally, we also train a RoBERTa sequence classification model from scratch, to assess if the pretraining step is necessary, refered to as the “base” model.

Our models share the architecture shown in table 3.2 that was found to be the optimal configuration after an extensive hyperparameter sweep of 90 runs. Training was performed with a single Nvidia A40 GPU, and 16GB of ram with a batch size of 128 for a single epoch, using the AdamW optimizer, and Cross-Entropy loss function provided by Pytorch.

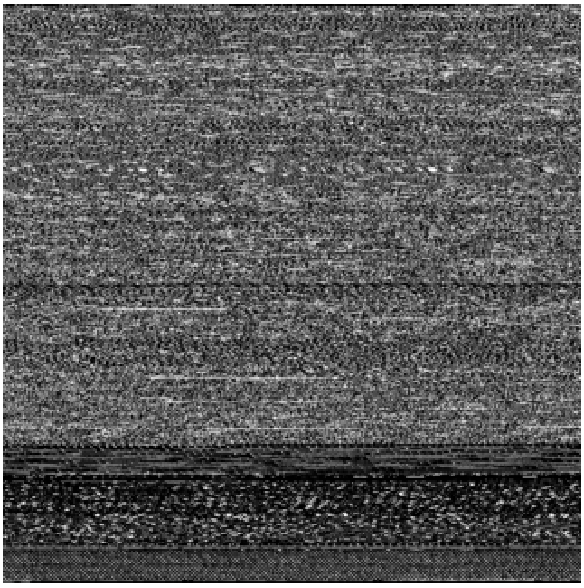
For each opcode sequence, the model predicts two values, the likelihood of the sequence being malicious, or benign where the larger of these two values is its predicted class. However, the full opcode sequence of a program is too long to be used as the models input. Therefore, we split the full sequence of opcodes into multiple sub-sequences of some fixed length, and aggregate the logits to classify a program. We test both raw logit mean aggregation, softmax logit mean aggregation, and majority voting, and found raw logit mean aggregation to be most effective.

Table 3.2: RoBERTa sequence classification model parameters

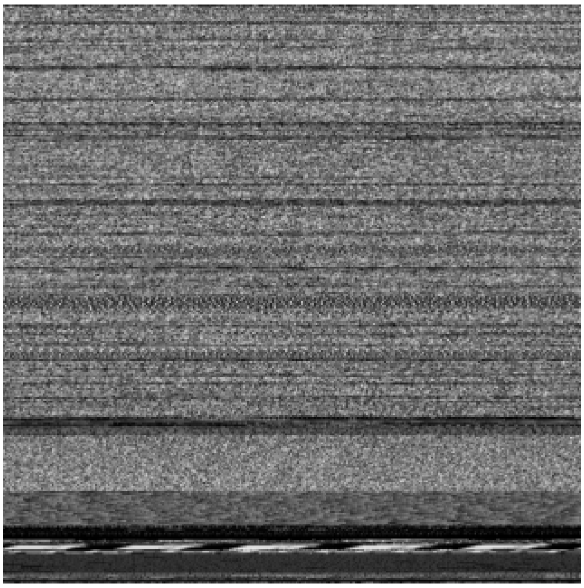
Parameter	**Value**
Sequence Length	32
Hidden Layer Size	512
Hidden Layers	7
Attention Heads	4
Intermediate Layer Size	2048
Activation Function	GELU
Hidden Dropout Rate	0.01
Attention Dropout Rate	0.001

3.5 CNN for Image Classification

In addition to the RoBERTa sequence classification model, we also evaluate an image based approach. We use the EfficientNet-B0 Convolutional Neural Network architecture (Tan et al., 2020), due to its computational efficiency and high performance on image classification problems. The model is



(a) Image representation of a benign executable.



(b) Image representation of a malicious executable.

Figure 3.5: How images can be used to represent programs.

trained from a random initialisation, and the final layer of the model is modified to output a binary class likelihood values. Programs are converted to greyscale images using the procedure outlined in appendix A.3, converted to RGB, and resized to 320×320 pixels before it is used by the model, as shown in fig. 3.5.

As previously established, the size of benign programs is typically much higher than malicious programs. Therefore, benign images are resized to a greater extent to that of malicious images. Resizing an image will introduce artifacts to the image, such as anti-aliasing or blurring. Thus, benign images may have more prevalent artifacts than malicious images which could be used by the model to learn a suboptimal solution. To prevent this from occurring, we train the same model on a subset of the dataset where the distribution of program size between classes are more similar, known as the "filtered" model. We use a genetic algorithm that minimises the Kullback-Leibler divergence D_{KL} between the distributions of program size for each class of program by modelling it as a Knapsack problem (Khuri et al., 1994). We also encourage the algorithm to select a subset with an approximately equal number of malicious and benign samples to ensure class balance.

We train both the base model, and the filtered model trained for 20 epochs, with the binary cross entropy loss function and a batch size of 32. Training was conducted using an Apple M2 Pro CPU, leveraging the metal performance shaders backend for hardware acceleration, with 16GB of RAM using the AdamW optimiser.

3.6 Obfuscation

In modern malware, obfuscation is highly pervasive. As a result of this, we believe that static malware detection methods learn to identify obfuscation as opposed to other malicious characteristics. However, obfuscation is not exclusive to malicious programs, as legitimate software developers may use similar techniques to protect their intellectual property. Should this be the case, legitimate benign software that have been obfuscated may be misidentified as malicious, thereby degrading the effectiveness of these approaches in real world deployment.

We investigate this by observing how the models performance changes as we increase the presence of obfuscation within the unobfuscated dataset. We perform three trials for each model, that vary which kinds of program are to be obfuscated. For example, only malicious, benign, and all programs. At each step, we obfuscate a certain percentage of programs within the dataset and this percentage increases by 10% upon completion. We collect the logits for each sequence, which are used to calculate both the sequence level, and the file level metrics. We also perform this experiment on the image classification model to see if these models are better suited to obfuscation resiliant malware detection.

Obfuscation is performed using a tool known as UPX (Markus F.X.J. Oberhumer et al., 2025), which is a binary packing tool. We chose this tool due to its extensive compatability with windows executables. Furthermore, a number of our malicious executables have been identified as using this tool for obfuscation, and it has also been used to compress and obfuscate benign files. Therefore, we feel that it is a suitable tool to replicate the forms of obfuscation used by both malicious and legitimate software developers seen in the wild without having to implement our own obfuscation tooling.

Chapter 4

Results

4.1 Baseline

Table 4.1: Performance of our baseline methods for malware detection based on the import tables of executables

Algorithm	Feature	Accuracy	F1 Score	Precision	Recall
Support Vector Classifier	DLL	0.89	0.89	0.89	0.89
	Function	0.92	0.92	0.92	0.92
Logistic Regression	DLL	0.87	0.87	0.87	0.87
	Function	0.91	0.91	0.92	0.91
Decision Tree	DLL	0.88	0.88	0.89	0.89
	Function	0.93	0.93	0.93	0.93

The results of our baseline approaches are presented in table 4.1, and indicate that all methods tested are comparible in terms of performance. When trained on non-unique external function usage, these methods achieve an F1 score of approximately 0.9. External library usage however, incurs a minor degradation in classification performance, as the same methods achieve an F1 score of approximately 0.88. Therefore, we can assume that the usage of external functions is a better indicator of a programs intent.

This can be explained by our earlier analysis of executable similarity. We found that the average pairwise similarity between malicious and benign executables based on the function level information was much lower than that of the DLL information. This indicates that malicious and benign samples overlap to some extent in terms of which external libraries are used by each. Function level information however demonstrates a lower similarity, suggesting that malicious and benign functions typically use different functions.

Our results indicate that the best algorithm for classification was the decision tree classifier, trained on external function information. In addition to providing the best performance of all baseline methods tested in this experiment, decision tree algorithms have the added benefit of being interpretable. This means that we can visualise the internal mechanisms of the algorithm, providing additional insight into what makes a file malicious according to this model.

4.2 Sequence Classification

Table 4.2: Performance of our RoBERTa model for opcode sequence classification and malware detection

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8756	0.8742	0.8798	0.8756	0.8976	0.8971	0.9095	0.8989
From Pretrained	0.9010	0.9002	0.9042	0.9010	0.9452	0.9452	0.9479	0.9458
From Pretrained (w/ Frozen Attention Weights)	0.8302	0.8265	0.8409	0.8302	0.8076	0.8037	0.8393	0.8098

In table 4.2 we present both the sequence level, and file level classifiction metrics of our machine-code langauge models. We find that in sequence level tasks, where the model is tasked with classifying a subsequence of opcodes as either malicious or benign, the pretrained model offers the best performance. When this pretraining stage is skipped, performance degrades by approximately 3%, indicating the prior understanding of opcode sequences is necessary to more accurately identify malicious sequences. In comparison to our baseline approaches presented in table 4.1, only the pretrained model offers similar performance.

In addition to this, the performance of the same pretrained model is impacted significantly when the parameters of its attention layers are frozen. These weights encode the understanding of langaue developed during its pretraining stage to create an encoded representation of a sequence, which is then used by a classification head to determine its classficiation. In fine-tuning tasks, these initial layer weights may be frozen to reduce the computational cost, and the amount of training data used during training the model. However, we see that freezing the attention layers has a significantly negative implication on the models performance.

These sequence level tasks however are not representative of our models performance in real world tasks. Therefore, to understand how they perform in real-world malware detection, we are required to reduce multiple sequence classifications across an entire program, into a single file-level classification. By taking the mean of each, un-normalised pair of class logits, we are able to find this file-level classification. From this, we find that the pretrained model achieves the best file-level performance, with an F1 score of 0.94, outperforming our baseline approaches by a small margin. This increase in performance between sequence level and file level tasks is also seen in the non-pretrained model to a lesser extent. However, the model with frozen attention layers exhibits a decrease between sequence level, and file level performance.

4.3 Image Classification

Table 4.3: Performance of the image classification models for malware detection

Dataset	Accuracy	F1-Score	Precision	Recall
Full	0.9316	0.9315	0.9314	0.9322
Filtered	0.9306	0.9306	0.9309	0.9306

Both image classification models demonstrate comparible performance to our baseline approaches, as shown in table 4.3. In addition to this, they also outperform both the non-pretrained and the frozen attention weight sequence classification models. However, they are both beaten by the pretrained sequence classification model in file level tasks. Images used by this model contains the entire

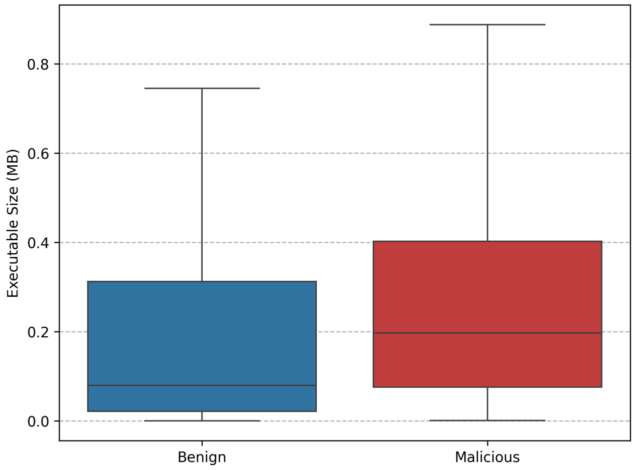


Figure 4.1: Distribution of executable size in the filtered dataset

executable, including the metadata stored in the executable header, static data, and the code itself. Our sequence classification models however, only use the opcodes of the instructions to determine a programs classification. Therefore, pretrained sequence classification models are more accurately identify malware, with less information, and by integrating this aforementioned information used by image classification models, we may be able to further improve the sequence classification models ability to recognise malicious programs.

We have also shown that there is a significant discrepancy in the size of executables across classes in our dataset. As we are required to resize images to a constant resolution of 320×320 pixels, artifacts originating from the resizing algorithm may be more pervasive in a single class. This could allow the model to develop an suboptimal solution to the problem as it may rely on the presence of these artifacts to distinguish between malicious and benign software. Therefore, we train another model which is trained on a subset of our original dataset, which has a more consistent distribution of executable sizes across classes. A genetic algorithm was used to identify this subset (appendix A.4), and the resulting collection has distributional distance of zero, and an approximately equal ratio of malicious to benign samples (fig. 4.1). However, the models trained on this dataset demonstrate performance in line with the model trained on the full dataset, demonstrating that this is not the case.

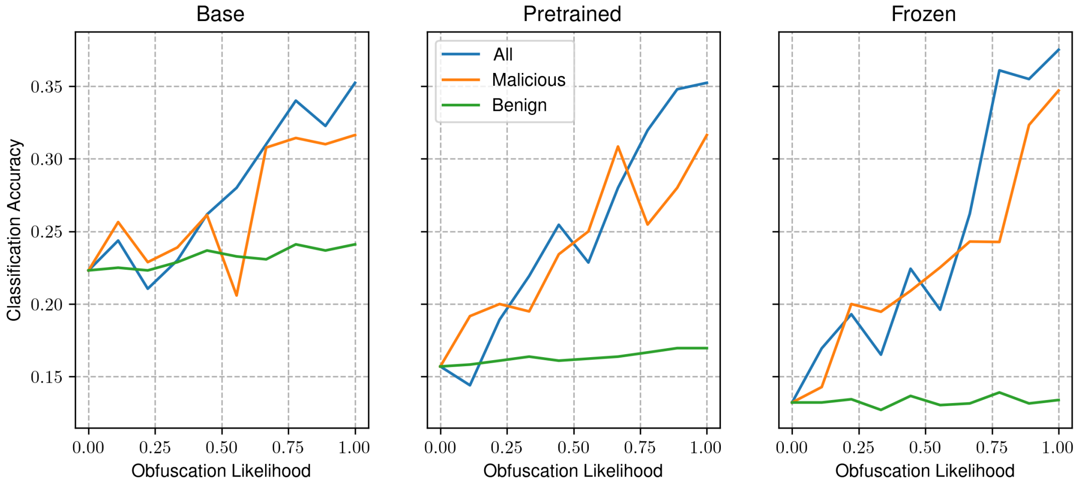
4.4 Obfuscation Experiment

4.4.1 Sequence Classification

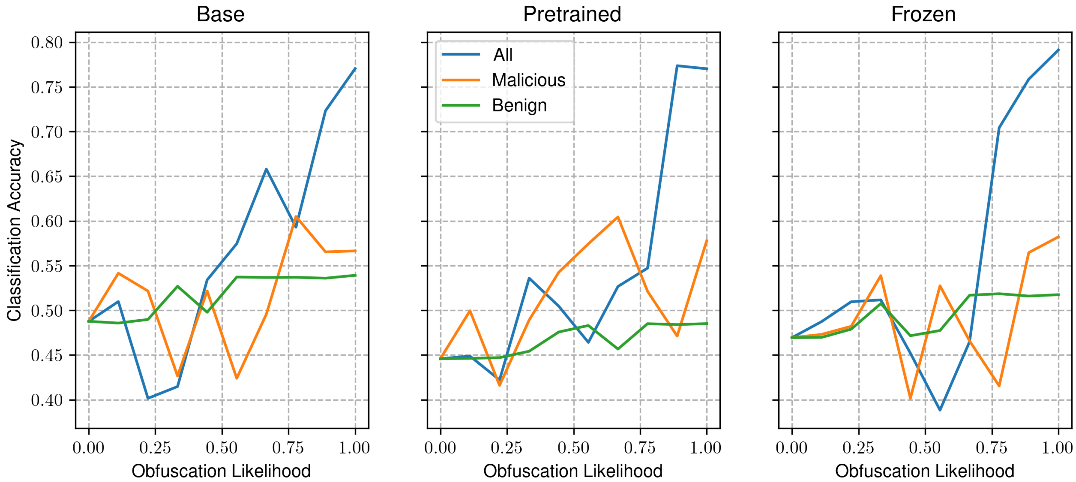
We find that our sequence classification models seem to heavily rely on obfuscation as an indicator of a programs intent. This can be seen in Figure 4.2a, where an increased presence of obfuscated malicious samples corresponds with an increase in classification accuracy. When only bengin files are obfuscated however, we see no meaningful trend.

However, when sequence level accuracy is considered, this trend is pronounced to a lesser extent, as shown in Figure 4.2b. We do see an a positive correlation between classification accuracy and obfuscation likelihood across all samples. However when obfuscation is applied to a single class of programs in the dataset, no correlation is seen.

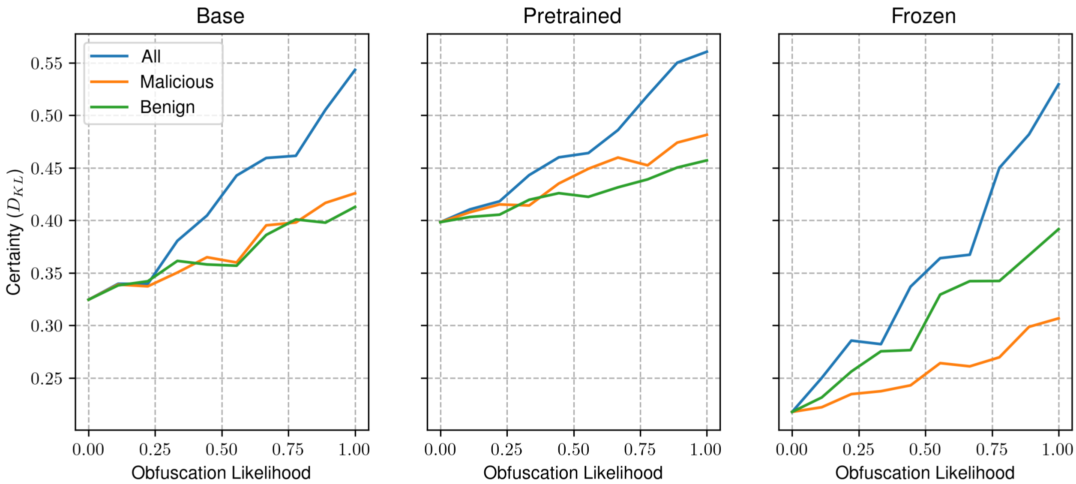
To better understand how our models handle obfuscation, we quantify the certainty in their predictions by taking the Kullback-Leibler Divergence eq. (4.1) between the models output, and the uniform distribution. This metric tells us how confident a model is in its predictions, by measuring



(a) File-level classification metrics



(b) Sequence level classification metrics



(c) Confidence in model predictions

Figure 4.2: Impact of obfuscation on the performance of file and sequence level classification, and certainty of the RoBERTa opcode sequence classification models.

the distributional difference between its predicted class likelihood distribution, and a uniform distribution representing the model picking at random. We find that certainty increases significantly across all trials as we increase the prevalance of obfuscation as shown in fig. 4.2c. This suggests that the model considers obfuscation as highly indicative of a samples class.

$$D_{KL}(p \parallel q) = \sum_{x \in \mathcal{X}} p(x) \log\left(\frac{p(x)}{q(x)}\right) \tag{4.1}$$

We also see that across all models tested in this experiment, the trends are largely similar. For example, our results show that as the presence of obfuscated malicious samples increase, the classification increases accordingly. This trend is seen in all of the models used in this experiment, however the extent of the trend varies between them. This suggests that our models are learning similar representations, yet are limited by other factors such as a reduced prior understanding of the langauge, and insufficient model complexity. To visualise this, we perform representational similarity analysis (RSA) on the activations of our models at each layer using Centered Kernel Alignment (CKA, shown in appendix A.5) as outlined in (Kornblith et al., 2019). This metric quantifies the correspondance between hidden representations of data across models trained from different initialisations. This is shown in fig. 4.3, and indicates that the similarity of representations in the attention layers of the pretrained and non-pretrained models decrease with the number of layers, and that the representations at the classification heads are very similar. Our frozen model however, demonstrates a higher extent of representational similarity in the attention layers, and significant disimilarity at the classification head.

We then repeat this experiment on two identical groups of models trained on modified datasets to see if data augmentation can produce models which are more resiliant to obfuscation. The first of these datasets has had all of its benign samples obfuscated prior to opcode extraction, and the second contains both the obfuscated and regular benign samples. We refer to these as our obfuscated bengin and partially obfuscated datasets respectively.

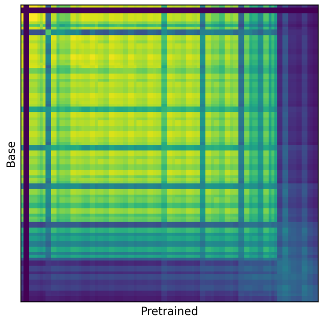
Obfuscated Benign

The models trained on the this dataset all demonstrate increased classification accuracy when only bengin samples are obfuscated. When only malicious programs are obfuscated however, its accuracy decreases. In addition to this, when all samples are targets for obfuscation, the file level accuracy remains relatively consistent at approximately 50%, suggesting random choice as shown in fig. 4.4a. Furthermore, the sequence classification metrics demonstrate that accuracy also remains consistent as we increase the number of obfuscated benign samples, whereas it decreases as more malicious samples are obfuscated. The certainty of the models in these predictions indicate that increasing the presence of obfuscation leads to greater uncertainty.

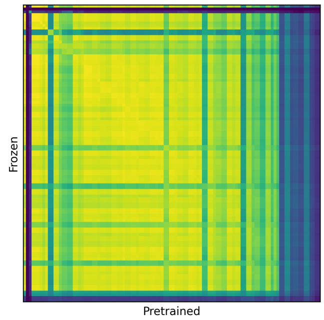
Partially Obfuscated Benign

Models trained on the partially obfuscated benign dataset demonstrate more consistent file-level classification accuracy when only malicious samples are obfuscated as opposed to the fully obfuscated benign models, which decrease under the same conditions. However, this is not the case for the frozen model, as we see a significant decrease in accuracy with increasing presence of obfuscated malicious samples.

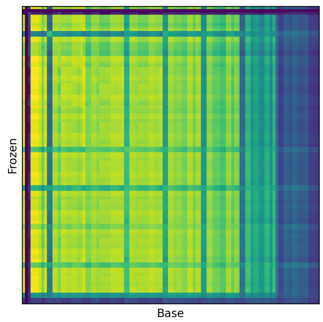
Furthermore, as was the case in the obfuscated benign models, accuracy does increase with the number of obfuscated benign samples in both the pretrained, and frozen models. The model trained from a random initialisation however, exhibits a unique trend in classification metrics between different obfuscation targets. In comparison to the other models tested in this experiment, where we see highly similar trends between different models trained on the same data, that differ only in their extent. However, in this case, we see that the classification accuracy is similar across obfuscation targets as opposed to the other models, which increase when all samples, and benign samples are



(a) RSA of the pretrained and non-pretrained models.



(b) RSA of the pretrained model and our pretrained model with frozen attention layer weights.



(c) RSA of the pretrained model with frozen attention layer weights and the non-pretrained model.

Figure 4.3: Representational Similarity Analysis (RSA) of our baseline models using Centered Kernel Alignment.

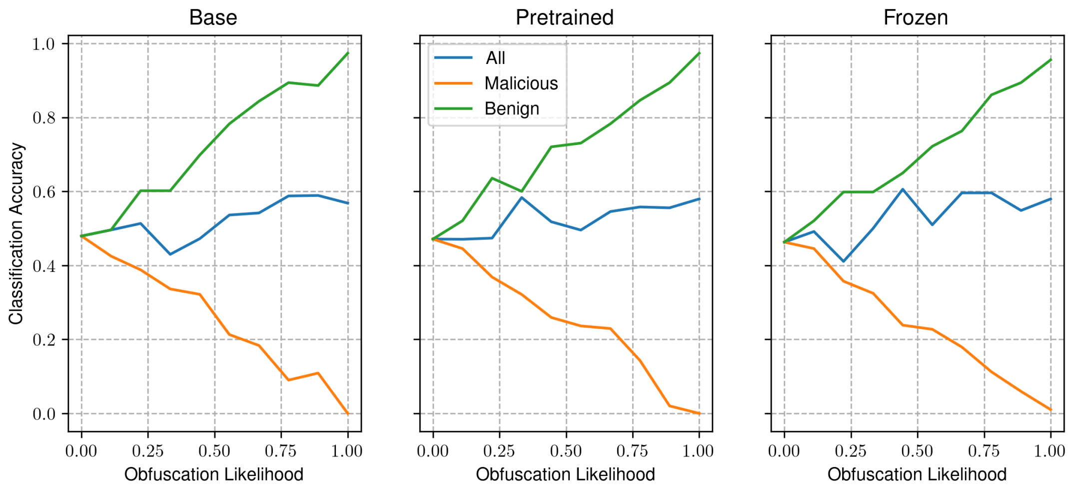
targets for obfuscation, while remaining consistent or decreasing with increased obfuscation likelihood.

However, sequence level accuracy tells a different story. Between all models, we see sequence level accuracy decrease as the number of obfuscated malicious samples increase. We also see that the observed increase in accuracy with the number of obfuscated benign samples is less pronounced than the file level metrics. As we increase the number of obfuscated samples in both classes, there is no discernable trend.

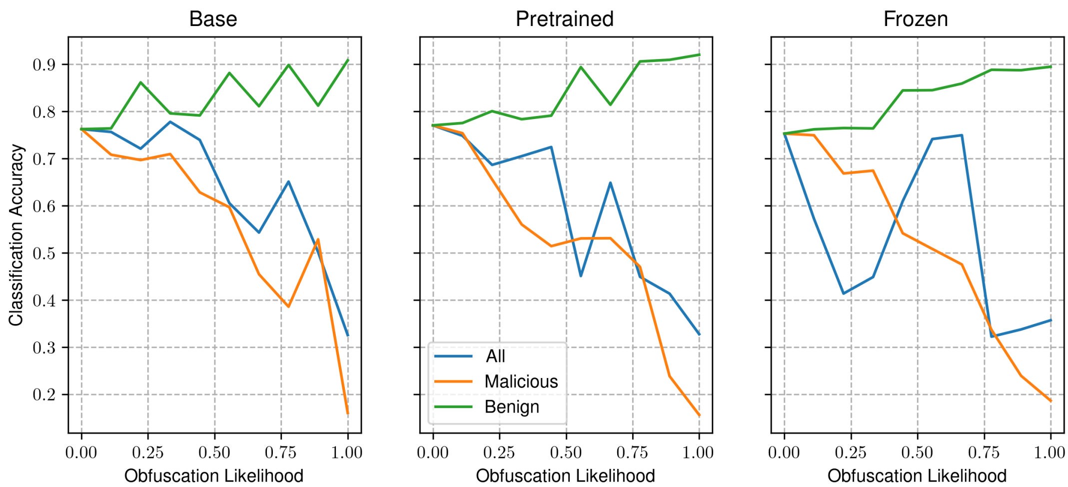
Furthermore, the KL divergence representing the models confidence, decreases with obfuscation likelihood. This can also be seen in the obfuscated benign models.

4.4.2 IMAGE CLASSIFER

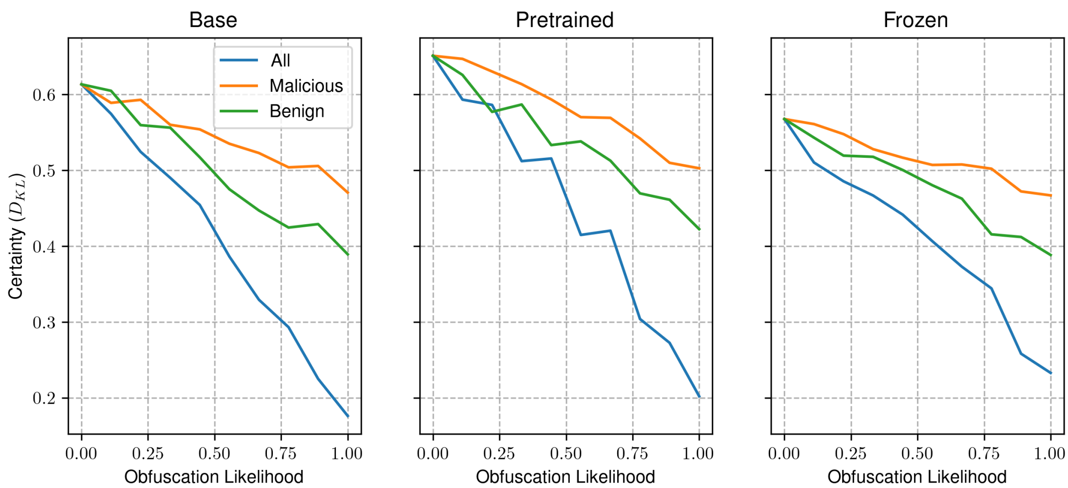
The impact of obfuscation on our image classification models is shown in Figure 4.6. We find that both of the image based malware detection models are more likely to identify malicious software based on the presence of obfuscation. In both cases, we see that the presence of obfuscation increases across our malicious samples is positively correlated with the classification accuracy. In comparison to this, an increased prevalance of obfuscation in benign samples leads to a degradation in performance. Furthermore, when obfuscation is applied across all samples in this dataset, we see no meaningful trend in classification accuracy as obfuscation becomes more common.



(a) File-level classification metrics

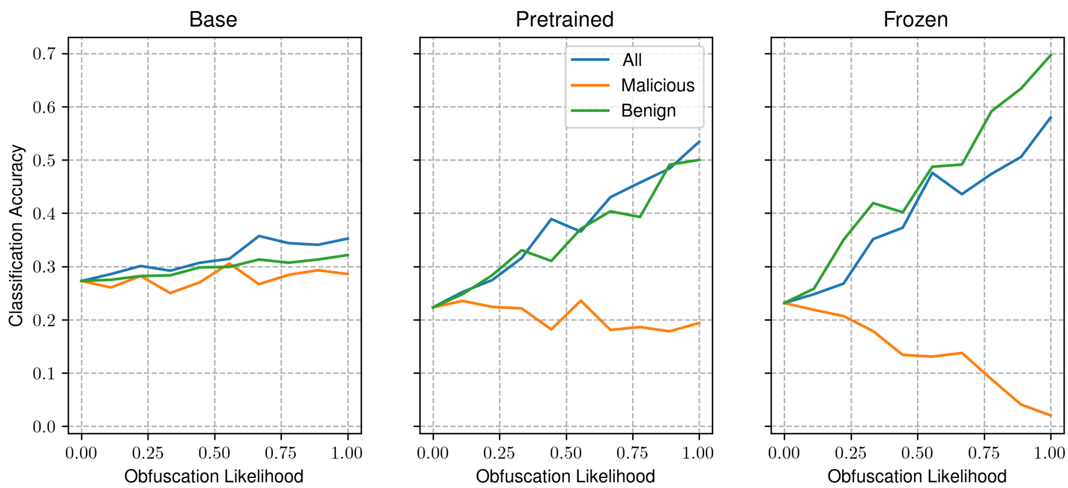


(b) Sequence level classification metrics

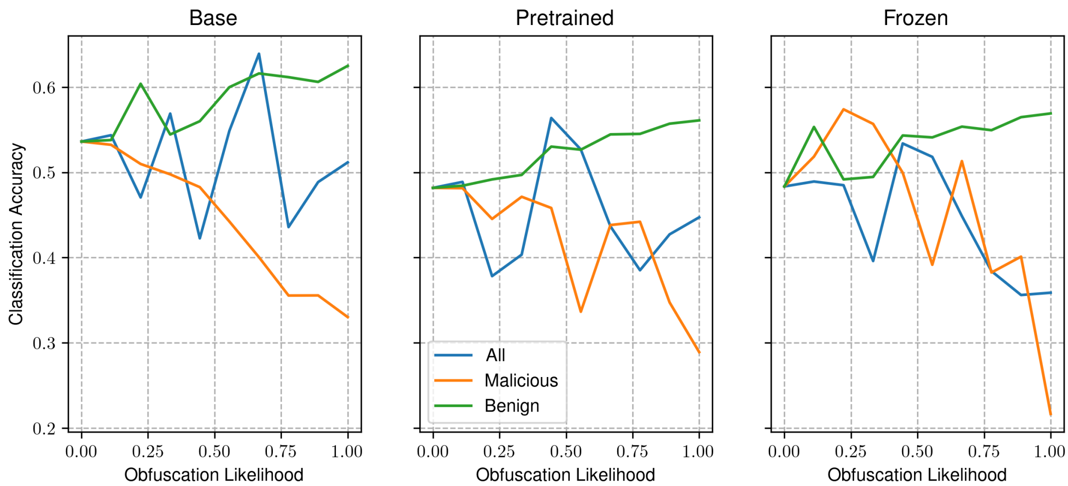


(c) Confidence in model predictions

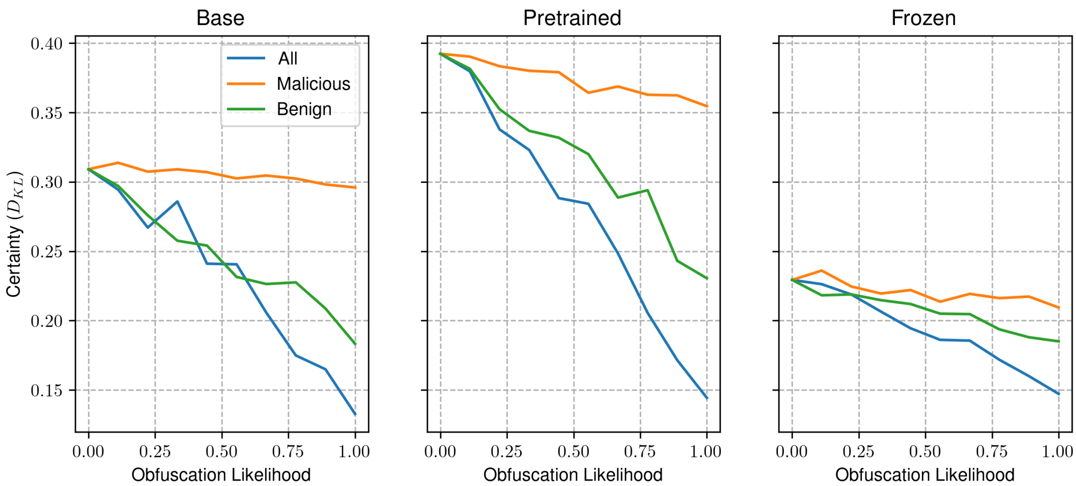
Figure 4.4: Impact of obfuscation on the performance of file and sequence level classification, and certainty of the RoBERTa opcode sequence classification models trained on the obfuscated bengin dataset.



(a) File-level classification metrics

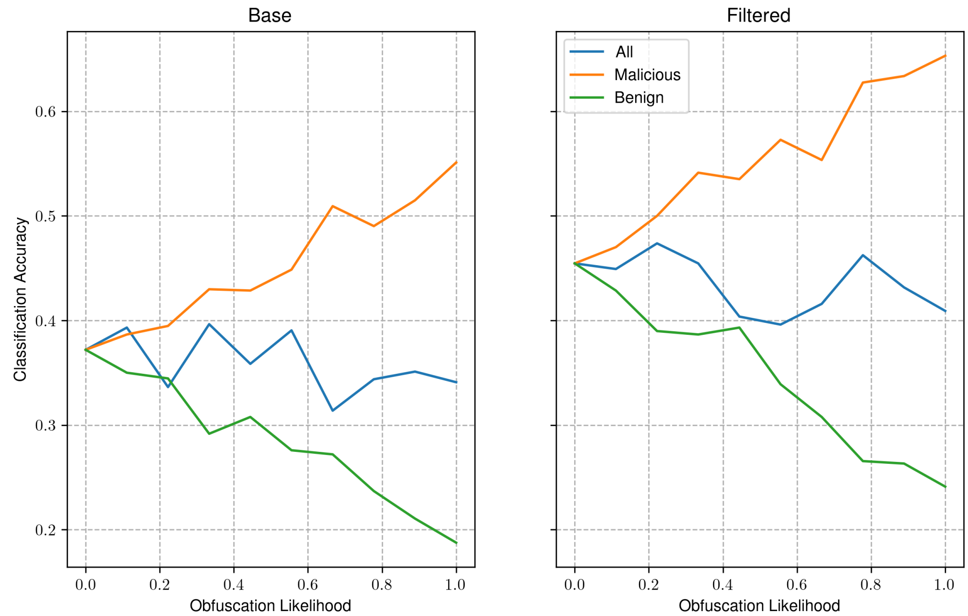


(b) Sequence level classification metrics

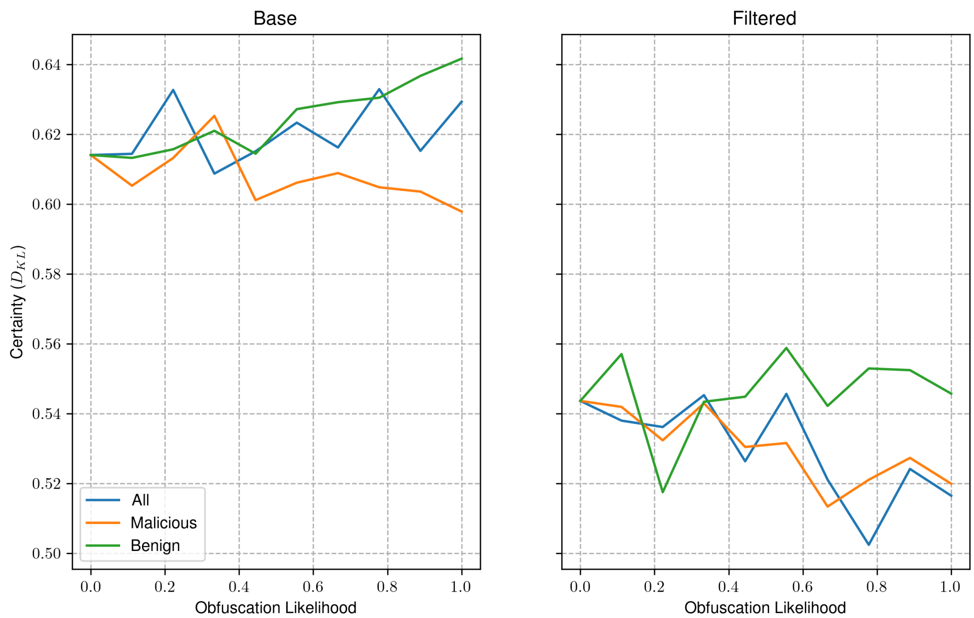


(c) Certainty (KL-Divergence between logits and uniform distribution)

Figure 4.5: Impact of obfuscation on the performance of file and sequence level classification, and certainty of the RoBERTa opcode sequence classification models trained on the partially obfuscated benign dataset.



(a) Classification accuracy



(b) Confidence in model predictions

Figure 4.6: Impact of obfuscation on the performance of image-based malware detection models, and the confidence of the model in its predictions.

Chapter 5

Discussion

In this research, we assess the suitablility of deep learning techniques for static malware detection, and the resiliance of these approaches to obfuscation. We evaluate three solutions to this problem, consisting of linear machine learning models, transformer based sequence classification models, and an image classification model using CNNs.

Linear models comprise our baseline against which all other approaches are compared. We test a logistic regression, decision trees, and support vector machine with a radial basis function (RBF) kernel. Each of these models are trained on information extracted from the import tables of programs in our dataset, informing the models as to which external functions and libraries are used by a program.

The sequence classification models uses modern natural language processing techniques, such as transformer based langauge models and pretraining. We use a RoBERTa, a BERT based architecture for langauge modelling, trained on sequences of assembly opcodes. We assess the impact of pretraining by training two sequence classification models, one using a pretrained masked language model, and another trained from a random initialisation. We also train another pretrained model which has had all parameters in its attention layers frozen to assess if the representations of the pretrained model are sufficient at identifying malicious opcode sequences.

Convolutional Neural Networks, specifically the EfficientNet architecture, offer a more computationally efficient approach trained on image representations of programs in our dataset. We test this model, and a model trained on a subset of the dataset with a more similar distribution in the size of programs between classes. This was conducted to determine if the model uses rescaling artifacts to distinguish between malicious and benign programs, as the latter are typically larger than malware.

We then test these models on a secondary dataset of un-obfuscated programs, and increase the presence of obfuscation in each class to see if the presence of obfuscation indicative of malware. In this chapter, we provide our interpretation of the results, its relation to the work of others, and the limitations of this investigation.

5.1 Interpretation of Results

Baseline

Our baseline approaches achieve an average classification accuracy of 90%. Between models trained on the usage of functions and libraries, we see an marginal increase in performance of approximately 4%. This is seen across each of the algorithms tested, which indicates that the external functions used by a programs are a better feature than external library usage. We believe that this is due to the difference in the amount of information encoded by their respective feature vectors. Across our entire dataset, we observe 196 external libraries, as opposed to 5926 external functions seen more than once. Therefore, the external function features encodes significantly more information, which

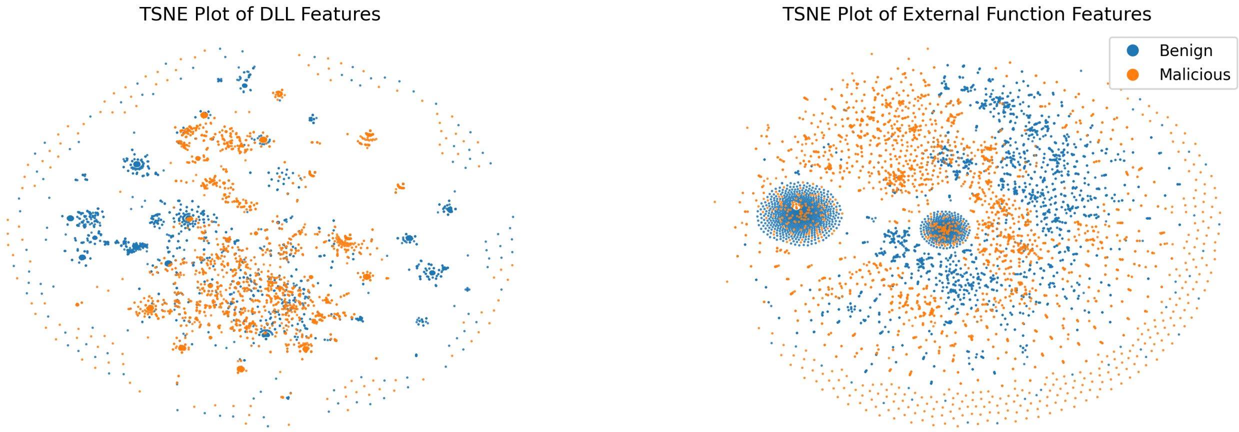


Figure 5.1: Comparison between the functions and DLLs used by malicious and benign programs.

allow the model to better understand malicious characteristics. We also find that DLLs are more commonly seen in both classes of programs when compared to external functions, as seen in fig. 5.1, and fig. 3.3. This suggests that the DLLs used by malicious programs, are similar to the ones used by benign programs, whereas the external functions are more telling of a programs intent.

Of these baseline methods, we find that the decision tree algorithm achieves the best classification performance of approximately 93%. In comparison to (Schultz et al., 2001), in which the DLL and function usage was used to identify novel malware with a rule based classification algorithm known as RIPPER, our results demonstrate improved performance. In this work, models using the DLL usage information achieved 83% accuracy, and 89% for function usage which seems to support our findings, despite being lower overall. We believe these results are an effect of the substantially smaller dataset, leading to decreased model performance. They also test RIPPER on a modified version of our features, where the DLL and function level information are combined, and the number of occurances throughout the program are counted, that achieves similar performance to the function level classifier.

We were unable to assess the resiliance of these models to obfuscation, as the tool used to produce obfuscated executables hides the import table used for feature extraction. Therefore, these models would be unable to classify UPX-obfuscated executables as their feature vectors would be empty.

Sequence Classification

The sequence classification models demonstrate performance in line with, and at times worse than the baseline approaches on sequence level tasks. For example, the pretrained model is able to identify malicious opcode sequences approximately 90% of the time. However, this increases to approximately 95% when we aggregate each of the opcode sequence logits across a file. This outperforms our baseline, nd demonstrates that this approach can accurately identify malware.

This performance can only be seen in the pretrained model however, as both the model trained from a random initialisation, and the pretrained model with frozen attention weights achieve accuracy in line with, and below the baseline respectively. In the case of the model trained from scratch, this may be due to insufficient training time, as the pretrained model has been trained for longer, because of the pretraining step. We see in fig. 4.3a that the representations at the initial layers are highly similar between the pretrained and non-pretrained models, which decreases with depth. It may be that with increased training time, the model approaches the solution of the pretrained model.

When the weights of the attention layers are frozen, allowing the model to modify only the classification head parameters, we see a drastic decrease in performance. We believe this is a result of reduced model complexity, as only approximately 1% of the models parameters are updated during training. It is well established that the number of a models learnable parameters is directly associated with its performance (Kaplan et al., 2020). By freezing the weights in the attention layers, the number of learnable parameters are reduced, leading to worse performance. Lee et al., 2019 shows that when fine-tuning a model, freezing of the first 75% of layers in similar models incurs only a 10% decrease in performance against its fully-trainable equivalent. Therefore, a better strategy for selecting which layers to freeze in fine-tuning tasks is necessary.

In terms of obfuscation resiliance, we find that each of these models demonstrate a high reliance on the presence of obfuscation as an indicator of malware. This can be seen in fig. 4.2, where we see accuracy increase with an increase in presence of obfuscated malicious samples. In direct comparison to this, as the presence of obfuscated benign samples increases, we see only a marginal increase in accuracy. Furthermore, we also see an increase in the confidence of the models predictions as obfuscation increases, which further supports our hypothesis.

We then train the same models on augmented data with both fully and partially obfuscated benign train samples. We find that data augmentation by introducing obfuscated benign samples is not sufficient to decrease a models' reliance on obfuscation. This is in direct contradiction to (Bacci et al., 2018), who suggest that the reliance of obfuscation on the predictions of static malware detection models can be mitigated by including obfuscated samples in its training corpora for android malware detection. However, the obfuscated benign models shown in fig. 4.4 demonstrates better accuracy with an increased presence of obfuscation in benign samples. However, when the same is done with malicious samples, the accuracy remains consistent. We believe this is because the same tool was used to generate these obfuscated samples in both the obfuscated benign dataset, and the experiment itself. The model has learned that characteristics of executables packed by UPX are indicative of benign samples, therefore as we increase the number of obfuscated benign samples, we see a corresponding increase in its accuracy. This implies that using a single form of obfuscation is insufficient to produce a dataset with forms of obfuscation representative of the forms used by the malicious samples in our dataset which may reduce the models reliance on obfuscation. We chose to use a single form of obfuscation, namely UPX, in both the datasets, and experiment due to its widespread support for windows executables. This allowed us to use a larger total number of samples in this experiment as would otherwise be possible. In comparison, (Bacci et al., 2018) use a custom obfuscation tool, which is more sophisticated. Future work could investigate the use other obfuscation tools, such as Alcatraz[1] or Lime[2].

When we train this model on the partially obfuscated benign dataset, we see a similar trend (Figure 4.5). However, as opposed to the fully obfuscated models, we see that accuracy increases when both only benign samples, and all samples are obfuscated, but a decrease when only malicious samples are obfuscated. This further indicates that the model has learned to identify UPX-style obfuscation as a potential indicator of benign executables. We also see a difference in characteristics between models trained on the same dataset, which is not seen across any of the other models or datasets. The model trained from a random initialisation exibits a consistent, marginal increase in accuracy across all obfuscation targets, whereas the others show a significant increase when both classes, and benign samples are targets. Similar characteristics across all other sequence classification models are not seen, as the trends between models are largely similar, and vary soley in the extent. This may indicate that the model has found a new solution to the problem, yet the trends in other metrics, such as sequence level accuracy and remain largely similar.

Furthermore, as opposed to models trained on the base dataset, the obfuscated benign, and partially obfuscated benign models demonstrate decreased confidence in their predictions. We see in fig. 4.2c, that as presence of obfuscated samples increases, the model becomes increasingly confident in its predictions. This suggests the model has identified it as an indicator of a programs class. The decreasing confidence in the other models however, imply that obfuscation negatively impacts the models predictions, as KL divergence between the the predicted class distribution, and the uniform

[1]https://github.com/weak1337/Alcatraz

[2]https://github.com/NYAN-x-CAT/Lime-Crypter

distribution approaches zero, indicating the model has assigned equal likelihood to both classes. Therefore, by introducing this form of obfuscation to the benign samples in our dataset, the model is less aware of what makes a sample malicious, and will struggle to reliably classify malware.

Image Classification

Both CNN malware detection models achieve similar performance approximately equal to the best of our baseline approaches. Therefore, these models do not achieve any notable benefit over these conventional approaches.

The model trained on the full dataset demonstrates a marginal increase in classification performance over the filtered dataset of approximately 0.1%. This suggests that rescaling artifacts have little impact on the solutions discovered by these models. However, as shown in fig. 5.2, there is very little difference between the average image-representations of each class in both datasets. Furthermore, we note that in both datasets, the benign samples demonstrate groups of lighter pixels at the same location. These may be artifacts originating from rescalling, which suggest that the genetic algorithm used to identify the subset is suboptimal and that further refinement is necessary.

We also note that the image classification approaches are significantly more vulnerable to obfuscation, as an increase in accuracy is seen as we increase the number of obfuscated malicious samples. However, as we increase the number of obfuscated benign samples, we see a decrease in the accuracy of the model. This suggests heavily that the model learns to identify obfuscation rather than other features indicative of malware.

5.2 Implications & Limitations

Implications

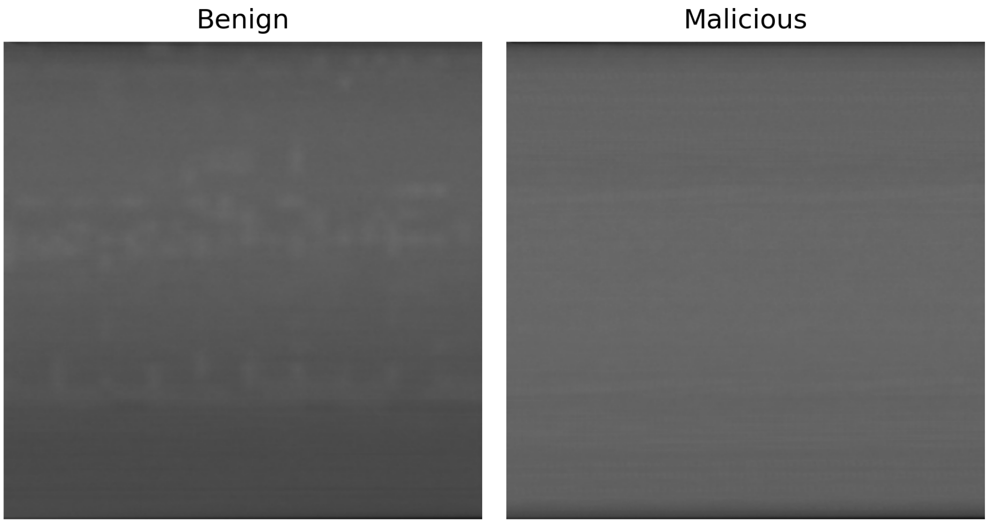
We believe that this research demonstrates the potential of natural language processing techniques for not only malware detection, but reasoning about executables as a whole. We have shown that language models are capable of identifying malicious sequences of instructions, based exclusively on opcodes. It may be that increasing the vocabulary, and scale of transformer models may be able to interpret and understand these executables in a similar manner to how large language models can understand nautral langauges.

Limitations

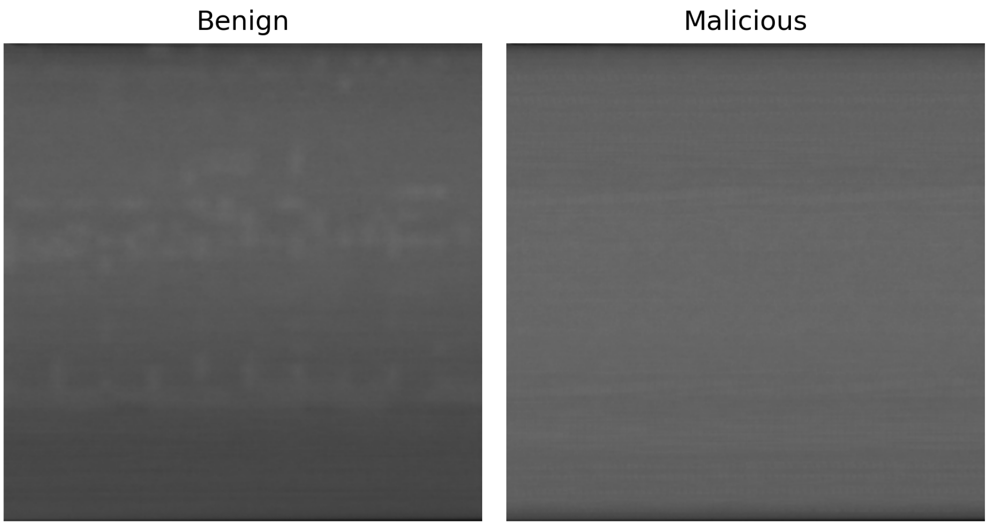
We identify a number of potential limitations of this work, which may have impacted the effectiveness of our work. To ensure this research is of high quality, we acknowledge these limitations, discuss how they impact the our findings, and how they may be prevented in future work.

The first of these issues relate to the data used to train, test, and conduct our obfuscation experiment. As explained in our literature review, there are no suitable large scale datasets containing executables for malware detection. Therefore, we curate our own dataset from executables found on the hardware of the researchers, and malware sample repositories. However, we were unable to gather a sufficiently large dataset. As shown in fig. 3.1c, we are limited to approximately 10000 samples, to ensure a balanced class distribution. It is well known that greater availability of training data is associated with greater model performance. Therefore, our work is limited by the availability of this data.

We have considered gathering benign samples from alternative sources, such as windows package repositories. However, these services often don't provide the executable, instead distributing an installer which would require extraction and installation by hand. Other sources used in the literature involve web scraping of sites such as github, and portable freeware. However, due to ethical and legal issues, we chose not to do this.



(a) Average representation of each class in the full dataset



(b) Average representation of each class in the filtered dataset

Figure 5.2

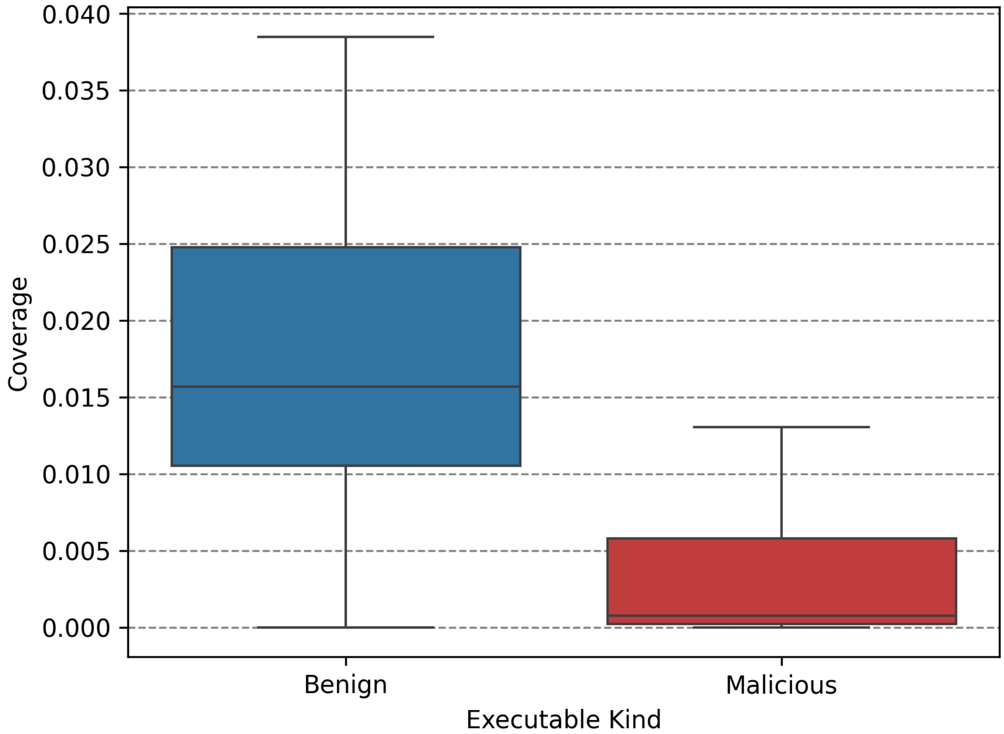


Figure 5.3: Extent of obfuscation between classes in the secondary dataset.

Furthermore, we postulate that our secondary dataset, used to conduct the investigation into obfuscation may not be sufficient for our purposes for a number of reasons. The first of these relate to the extent of obfuscation, we assume that the executables within this dataset are free of obfuscation. However, when applied to this dataset the static code coverage analysis, which was previously used to determine the extent of obfuscation in our main dataset demonstrates a greater disparity between classes as shown in fig. 5.3. This exceeds the disparity in the primary dataset, and either indicates that code analysis coverage may not be suitable for estimating obfuscation extent within a dataset, or that the malicious executables do in fact include obfuscation. Furthermore, our analysis of the github repositories of the malware found in this dataset demonstrate a lack of diversity. We note that a majority of these samples are simple proof of concept projects[3], and that keyloggers are overrepresented in this dataset.

During the obfuscation experiment, we were only able to perform a single run per experiment. This means that the results may be unrepresentative of the characteristics of the approaches tested in this research. Therefore, the obfuscation experiment should be run multiple times to ensure the presence of outliers in our results are mitigated. We were unable to conduct these repreated runs due to time limitations, as a single run of the experiment took approximately an hour per model, and obfuscation target. We acknowledge that this is a limitation in our implementation, as the obfuscation and opcode extraction occurs at each sample. To make the experiment more efficient, we suggest creating a copy of the dataset, where the opcodes of obfuscated executables are extracted.

Future Work

Future work should consider using a larger vocabulary size, and training these models on the full disassembly as opposed to the opcode sequences. Tokenization strategies such as WordPiece, which was used in the original BERT model (Devlin et al., 2019), may allow for a better representation of

[3]As these projects are publicly listed on github, we are able to distribute the sources of these samples, which can be found in our GitHub repository

the disassembly. We have considered using this information, however due to compute limitations, we deemed using the full disassembly to be unfeasible.

Additionally, the impact of obfuscation on the performance of dynamic models should also be tested. We have seen a number of obfuscation techniques that target these dynamic approaches, such as (Li et al., 2023), which prevent system API calls from being observed by the host which can prevent the extraction of API call sequences. These features are widely used in dynamic malware detection approaches seen in the literature, so this form of obfuscation may become more prevalent as malware developers become aware of these techniques. Therefore, ensuring these models are resiliant to forms of obfuscation targeting dynamic features is of high importance.

5.3 Conclusion

In this investigation, we explore the use of deep learning techniques for static malware detection, and the resiliance of these approaches to obfuscation. Our sequence classification model, based on the BERT architecture, identifies malicious opcode sequences, which are then be aggregated across an executables disassembly to produce the file level classification. We find that this model is able to outperform our baseline approaches by a small margin, indicating that techniques from natural langauge processing are suitable in this domain, despite using a reduced model size. We also find that other natural language processing techniques, such as pretraining with masked langauge modelling has a positive effect on the performance of these models. Furthermore, we find that this approach is reliant on obfuscation to determine a programs classification, which may lead to obfuscated benign samples being misidentified as malicious in real world usage. Augmentation of training data to include obfuscated benign samples has little impact on the performance of these models under such conditions, which indicates that the forms of obfuscation found in our malicious dataset differ from the forms used in our experiments.

Image classification approaches, offer a more computationally efficient approach. However, their performance is in line with that of our baseline. While they are more resiliant to obfuscation than the baseline approaches, as UPX hides the import table used to extract features for our baseline models, they demonstrate an increased reliance on such features for detecting malicious samples. This suggests that in real world deployment, these models would be more likely to misidentify obfuscated benign samples as malicious. Furthermore, the expected reliance of these models on artifacts originating from rescaling due to the discrepancy in size between malicious and benign samples is seemingly not an issue in this case.

Bibliography

A. Saeed, Imtithal, Ali Selamat and Ali M. A. Abuagoub (Apr. 2013). 'A Survey on Malware and Malware Detection Systems'. In: *International Journal of Computer Applications* 67.16, pp. 25–31. ISSN: 09758887. DOI: 10.5120/11480-7108. (Visited on 29/07/2025).

Aboaoja, Faitouri A. et al. (Jan. 2022). 'Malware Detection Issues, Challenges, and Future Directions: A Survey'. In: *Applied Sciences* 12.17, p. 8482. ISSN: 2076-3417. DOI: 10.3390/app12178482. (Visited on 10/12/2024).

Ainapure, Kaushik and Microsoft (Jan. 2025). *Dynamic Link Library (DLL)*. https://learn.microsoft.com/en-us/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library. (Visited on 15/08/2025).

Akbanov, Maxat, Vassilios G Vassilakis and Michael D Logothetis (2019). 'WannaCry Ransomware: Analysis of Infection, Persistence, Recovery Prevention and Propagation Mechanisms'. In: *Journal of Telecommunications and Information Technology* 1, pp. 113–124.

Alenezi, Mohammed et al. (Dec. 2020). 'Evolution of Malware Threats and Techniques: A Review'. In: *International Journal of Communication Networks and Information Security* 12, p. 326. DOI: 10.17762/ijcnis.v12i3.4723.

Anderson, Hyrum S. and Phil Roth (Apr. 2018). *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models*. DOI: 10.48550/arXiv.1804.04637. arXiv: 1804.04637 [cs]. (Visited on 05/08/2025).

Aslan, Ömer and Abdullah Asim Yilmaz (2021). 'A New Malware Classification Framework Based on Deep Learning Algorithms'. In: *IEEE Access* 9, pp. 87936–87951. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3089586. (Visited on 28/07/2025).

Attaluri, Srilatha, Scott McGhee and Mark Stamp (May 2009). 'Profile Hidden Markov Models and Metamorphic Virus Detection'. In: *Journal in Computer Virology* 5.2, pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. (Visited on 25/02/2025).

AV-TEST - The Independent IT-Security Institute (2025). *AV-ATLAS - Malware & PUA*. https://portal.av-atlas.org/malware. (Visited on 19/12/2024).

Bacci, Alessandro et al. (2018). 'Impact of Code Obfuscation on Android Malware Detection Based onStatic and Dynamic Analysis'. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Funchal, Madeira, Portugal: SCITEPRESS - Science and Technology Publications. DOI: 10.5220/0006642503790385. (Visited on 21/07/2025).

Baezner, Marie and Patrice Robin (Oct. 2017). *Stuxnet*. Report 4. ETH Zurich. DOI: 10.3929/ethz-b-000200661. (Visited on 22/07/2025).

Balakrishnan, Arini and Chloe Schulze (Dec. 2005). 'Code Obfuscation Literature Survey'. In.

Bridge, Karl and Microsoft (July 2025). *PE Format*. https://learn.microsoft.com/en-us/windows/win32/debug/pe-format. (Visited on 15/08/2025).

British Computing Society (June 2022). *BCS Code of Conduct*. https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/. (Visited on 14/08/2025).

Christodorescu, Mihai and Somesh Jha (Mar. 2004). 'Static Analysis of Executables to Detect Malicious Patterns'. In: 12.

Cochran, Kodi A. (2024). 'The CIA Triad: Safeguarding Data in the Digital Realm'. In: *Cybersecurity Essentials: Practical Tools for Today's Digital Defenders*. Ed. by Kodi A. Cochran. Berkeley, CA:

Apress, pp. 17–32. ISBN: 979-8-8688-0432-8. DOI: 10.1007/979-8-8688-0432-8_2. (Visited on 22/07/2025).
Damodaran, Anusha et al. (Feb. 2017). 'A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection'. In: *Journal of Computer Virology and Hacking Techniques* 13.1, pp. 1–12. ISSN: 2263-8733. DOI: 10.1007/s11416-015-0261-z. (Visited on 29/07/2025).
Darem, Abdulbasit et al. (Dec. 2021). 'Visualization and Deep-Learning-Based Malware Variant Detection Using OpCode-level Features'. In: *Future Generation Computer Systems* 125, pp. 314–323. ISSN: 0167-739X. DOI: 10.1016/j.future.2021.06.032. (Visited on 28/07/2025).
Demırcı, Denız et al. (2022). 'Static Malware Detection Using Stacked BiLSTM and GPT-2'. In: *IEEE Access* 10, pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. (Visited on 25/02/2025).
Devlin, Jacob et al. (May 2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. (Visited on 28/02/2025).
European Union Agency for Cybersecurity (2024). *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office. (Visited on 21/02/2025).
Habibi, Omar, Mohammed Chemmakha and Mohamed Lazaar (Aug. 2023). 'Performance Evaluation of CNN and Pre-trained Models for Malware Classification'. In: *Arabian Journal for Science and Engineering* 48.8, pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. (Visited on 25/02/2025).
Idika, Mathur (Feb. 2007). 'A Survey of Malware Detection Techniques'. In.
Jain, M., W. Andreopoulos and M. Stamp (2020). 'Convolutional Neural Networks and Extreme Learning Machines for Malware Classification'. In: *Journal of Computer Virology and Hacking Techniques* 16.3, pp. 229–244. DOI: 10.1007/s11416-020-00354-y.
Kakisim, Arzu Gorgulu, Sibel Gulmez and Ibrahim Sogukpinar (Mar. 2022). 'Sequential Opcode Embedding-Based Malware Detection Method'. In: *Computers & Electrical Engineering* 98, p. 107703. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2022.107703. (Visited on 28/07/2025).
Kamboj, Akshit et al. (Mar. 2023). 'Detection of Malware in Downloaded Files Using Various Machine Learning Models'. In: *Egyptian Informatics Journal* 24.1, pp. 81–94. ISSN: 1110-8665. DOI: 10.1016/j.eij.2022.12.002. (Visited on 28/07/2025).
Kaplan, Jared et al. (Jan. 2020). *Scaling Laws for Neural Language Models*. DOI: 10.48550/arXiv.2001.08361. arXiv: 2001.08361 [cs]. (Visited on 13/02/2025).
Karbab, ElMouatez Billah et al. (Mar. 2018). 'MalDozer: Automatic Framework for Android Malware Detection Using Deep Learning'. In: *Digital Investigation* 24, S48–S59. ISSN: 1742-2876. DOI: 10.1016/j.diin.2018.01.007. (Visited on 20/11/2024).
Khuri, Sami, Thomas Bäck and Jörg Heitkötter (Apr. 1994). 'The Zero/One Multiple Knapsack Problem and Genetic Algorithms'. In: *Proceedings of the 1994 ACM Symposium on Applied Computing*. SAC '94. New York, NY, USA: Association for Computing Machinery, pp. 188–193. ISBN: 978-0-89791-647-9. DOI: 10.1145/326619.326694. (Visited on 04/08/2025).
Ki, Youngjoon, Eunjin Kim and Huy Kang Kim (June 2015). 'A Novel Approach to Detect Malware Based on API Call Sequence Analysis'. In: *International Journal of Distributed Sensor Networks* 11.6, p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. (Visited on 02/11/2024).
Kornblith, Simon et al. (July 2019). *Similarity of Neural Network Representations Revisited*. DOI: 10.48550/arXiv.1905.00414. arXiv: 1905.00414 [cs]. (Visited on 12/08/2025).
Kramer, Simon and Julian C. Bradfield (May 2010). 'A General Definition of Malware'. In: *Journal in Computer Virology* 6.2, pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. (Visited on 21/02/2025).
Lee, Jaejun, Raphael Tang and Jimmy Lin (Nov. 2019). *What Would Elsa Do? Freezing Layers During Transformer Fine-Tuning*. DOI: 10.48550/arXiv.1911.03090. arXiv: 1911.03090 [cs]. (Visited on 12/08/2025).

Li, Ce et al. (Nov. 2022). 'DMalNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning'. In: *Computers & Security* 122, p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. (Visited on 27/11/2024).

Li, Yang et al. (Jan. 2023). 'APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity'. In: *Applied Sciences* 13.16, p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. (Visited on 25/02/2025).

M., Gopinath and Sibi Chakkaravarthy Sethuraman (Feb. 2023). 'A Comprehensive Survey on Deep Learning Based Malware Detection Techniques'. In: *Computer Science Review* 47, p. 100529. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2022.100529. (Visited on 20/11/2024).

Maniriho, Pascal, Abdun Naser Mahmood and Mohammad Jabed Morshed Chowdhury (Mar. 2024). 'A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions'. In: *Journal of Systems and Software* 209, p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. (Visited on 18/02/2025).

Markus F.X.J. Oberhumer, Laszlo Molnar and John F. Reiser (Aug. 2025). *UPX*. UPX - the Ultimate Packer for eXecutables. (Visited on 19/08/2025).

Merabet, Hoda El and Abderrahmane Hajraoui (2019). 'A Survey of Malware Detection Techniques Based on Machine Learning'. In: *International Journal of Advanced Computer Science and Applications* 10.1. ISSN: 21565570, 2158107X. DOI: 10.14569/IJACSA.2019.0100148. (Visited on 20/11/2024).

National Audit Office (NAO) (Oct. 2017). *Investigation: WannaCry Cyber Attack and the NHS*. Tech. rep. London, England: National Audit Office.

Ni, Sang, Quan Qian and Rui Zhang (Aug. 2018). 'Malware Identification Using Visualization Images and Deep Learning'. In: *Computers & Security* 77, pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/j.cose.2018.04.005. (Visited on 21/02/2025).

Quiquet, François (Oct. 2023). *An analysis of the Viasat cyber attack with the MITRE ATT&CK® framework*. (Visited on 21/07/2025).

Radare Organization (July 2025). *Radare2*. radare org. (Visited on 30/07/2025).

Rahali, Abir and Moulay A. Akhloufi (Oct. 2021). 'MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers'. In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. (Visited on 25/02/2025).

Rizin Organization (July 2025). *Rizin*. Rizin Organization. (Visited on 30/07/2025).

Rokon, Md Omar Faruk et al. (May 2020). *SourceFinder: Finding Malware Source-Code from Publicly Available Repositories*. DOI: 10.48550/arXiv.2005.14311. arXiv: 2005.14311 [cs]. (Visited on 31/07/2025).

Runwal, Neha, Richard M. Low and Mark Stamp (May 2012). 'Opcode Graph Similarity and Metamorphic Detection'. In: *Journal in Computer Virology* 8.1, pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. (Visited on 25/02/2025).

Schultz, M.G. et al. (May 2001). 'Data Mining Methods for Detection of New Malicious Executables'. In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. (Visited on 21/02/2025).

Sharma, Arindam, Pasquale Malacaria and MHR Khouzani (June 2019). 'Malware Detection Using 1-Dimensional Convolutional Neural Networks'. In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 247–256. DOI: 10.1109/EuroSPW.2019.00034. (Visited on 28/07/2025).

Singh, Jagsir and Jaswinder Singh (May 2020). 'Detection of Malicious Software by Analyzing the Behavioral Artifacts Using Machine Learning Algorithms'. In: *Information and Software Technology* 121, p. 106273. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106273. (Visited on 28/07/2025).

Tan, Mingxing and Quoc V. Le (Sept. 2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. DOI: 10.48550/arXiv.1905.11946. arXiv: 1905.11946 [cs]. (Visited on 31/07/2025).

Tian, Ronghua et al. (2009). 'An Automated Classification System Based on the Strings of Trojan and Virus Families'. In: *2009 4th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 23–30. DOI: 10.1109/MALWARE.2009.5403021.

Vaswani, Ashish et al. (Aug. 2023). *Attention Is All You Need*. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 30/07/2025).

Wang, Haojun et al. (2021). 'Deep Learning and Regularization Algorithms for Malicious Code Classification'. In: *IEEE Access* 9, pp. 91512–91523. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3090464. (Visited on 28/07/2025).

Yang, Limin et al. (May 2021). 'BODMAS: An Open Dataset for Learning Based Temporal Analysis of PE Malware'. In: *2021 IEEE Security and Privacy Workshops (SPW)*. San Francisco, CA, USA: IEEE, pp. 78–84. ISBN: 978-1-6654-3732-5. DOI: 10.1109/SPW53761.2021.00020. (Visited on 05/08/2025).

You, Ilsun and Kangbin Yim (2010). 'Malware Obfuscation Techniques: A Brief Survey'. In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.

Appendix A

Supplimental Information

A.1 Code Availability

All of the code used in this investigation are publicly available on our GitHub repository[1]. The dataset can be made available upon request, however due to copyright limitations we are unable to distribute benign samples.

A.2 Disassembly Extraction and Processing

Algorithm 1 Generate and extract disassembly from an executable

Input: Path to executable P
Output: Sequence of disassembled instructions of P

$L \leftarrow$ empty list
Load P into radare2
Perform full analysis ▷ R2 Command: `aaa`
if Instruction set of P is not x86 **then**
    return
end if
$S \leftarrow$ list of sections in P ▷ R2 Command: `iS`
$S_x \leftarrow$ filter S for executable sections
for all $s \in S_x$ **do**
    $o \leftarrow$ section offset of s
    $n \leftarrow$ section size of s
    $D \leftarrow$ disassemble bytes from o to $o + n$ ▷ R2 Command: `pda` o `@` $o + n$
    $D' \leftarrow$ filtered D to remove padding bytes
    Append D' to L
end for
return L

[1] https://github.com/Henry-Ash-Williams/masters-thesis

Algorithm 2 Extract opcodes from disassembly

Input: List of assembly instructions I
Output: Sequence of opcodes

$L \leftarrow$ empty list
for all $i \in I$ **do**
    $i' \leftarrow$ split i on spaces
    Append i'_0 to L
end for
return L

A.3 Conversion of Executable Programs to Greyscale Images

Algorithm 3 Convert a program to an image

Input: Program $P \in [0, 255]^L$ of L bytes
Output: Matrix $M \in [0, 255]^{d \times d}$

$d \leftarrow \lceil \sqrt{L} \rceil$
Initialize $V \in [0, 255]^{d^2}$ with zeros
for $i \leftarrow 1$ to L **do**
    $V_i \leftarrow P_i$
end for
Reshape V into matrix $M \in [0, 255]^{d \times d}$ in row-major order
return M

A.4 Genetic Algorithm

Algorithm 4 Apply a random mutation to a genotype

Input: Genome $G \in \{0, 1\}^N$ of N binary values, mutation rate $0 < p \leq 1$
Output: Mutated Genome $G' \in \{0, 1\}^N$

$V \in \mathbb{R}^N \leftarrow$ a vector of random numbers in $[0, 1]$
$M \in \{0, 1\}^N \leftarrow$ a binary vector where $V_i < p$
$G' \leftarrow G \oplus M$
return G'

Algorithm 5 Compute the fitness of a genotype

Input: Genome $G \in \{0,1\}^N$ of N binary values
Output: The fitness $f \in \mathbb{R}$ of G

$N_m \leftarrow$ Number of malicious samples in G
$N_b \leftarrow$ Number of benign samples in G
$\mu_m, \sigma_m \leftarrow$ mean and variance of the size of malicious programs in G
$\mu_b, \sigma_b \leftarrow$ mean and variance of the size of benign programs in G
$d \leftarrow$ the Kullback-Leibler Divergence between $p_m \sim \mathcal{N}(\mu_m, \sigma_m)$ and $p_b \sim \mathcal{N}(\mu_b, \sigma_b)$
$r \leftarrow \text{clip}\left(1 - \frac{N_b}{N_m}, 0, 1\right)$
$w_d \leftarrow -\left(\frac{e^{-d}}{1+e^{-d}}\right)$
$w_r \leftarrow 1 - w_d$
$f \leftarrow w_d d + w_r r$
return f

Algorithm 6 Simulate evolution over a population

Input: The fittest member of the previous population $G \in \{0,1\}^N$, Population size N_p, Mutation rate p_m
Output: The fittest member of the current population $G' \in \{0,1\}^N$

$P \leftarrow N_p$ Copies of G
$F_P \leftarrow$ Fitness of current population ▷ Algorithm 5
$O \leftarrow$ Mutated population of offspring from P with some mutation rate p_m ▷ Algorithm 4
$F_O \leftarrow$ Fitness of offspring population ▷ Algorithm 5
$P' \leftarrow$ New population where the fittest of a parent and its offspring survives
$G' \leftarrow$ The fittest member of P'
return G'

Algorithm 7 Simulate the process of evolution over multiple generations

Input: Number of generations to simulate N_g
Output: The fittest member of the last generation G

$G_0 \in \{0,1\}^N \leftarrow$ initial random genotype
for $i \leftarrow 0$ to N_g **do**
    $G_{i+1} \leftarrow$ Simulate a population with G_i ▷ Algorithm 6
    $F_i \leftarrow$ Fitness of G_{i+1} ▷ Algorithm 5
    if F_i has not decreased in the past 10 generations **then**
        return G_{i+1}
    end if
end for
return G_i

A.5 Centered Kernel Alignment

Given two matrices of n examples of the output from layers in two neural networks trained from different initialisations, $\mathbf{X} \in \mathbb{R}^{n \times d_x}$, and $\mathbf{Y} \in \mathbb{R}^{n \times d_y}$, the Centered Kernel Alignment between these representations is defined as follows:

$$\mathrm{CKA}(\mathbf{K},\mathbf{L}) = \frac{\mathrm{HSIC}(\mathbf{K},\mathbf{L})}{\sqrt{\mathrm{HSIC}(\mathbf{K},\mathbf{K}) \cdot \mathrm{HSIC}(\mathbf{L},\mathbf{L})}} \tag{A.1}$$

Where HSIC refers to the Hilbert-Schmidt independence criterion, defined in eq. (A.2), $\mathbf{K} = \mathbf{X}\mathbf{X}^T$, and $\mathbf{L} = \mathbf{Y}\mathbf{Y}^T$.

$$\mathrm{HSIC}(\mathbf{K},\mathbf{L}) = \frac{\mathrm{tr}(\mathbf{KHLH})}{(n-1)^2} \tag{A.2}$$

Where $\mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{J}_n$, and $\mathbf{J}_n$ is an $n \times n$ matrix of ones.

A.6 Windows Portable Executable File Format

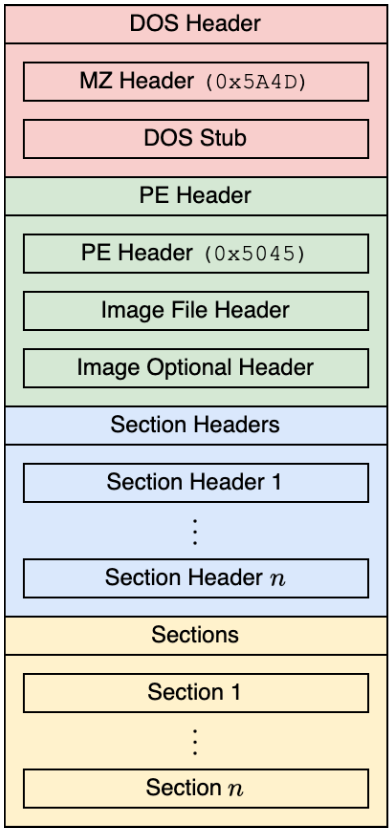


Figure A.1: A high level overview of the windows portable executable file format.

Windows executables are defined in the portable executable file format. These files are not written by developers themselves as they require expert knowledge of low level programming, and instead are generated by a compiler. A high level overview of this file format is shown in Figure A.1, and it has been in use since Windows 95, making it incredibly widespread throughout the internet.

The file begins with a DOS (Disk Operating System) header for compatibility reasons (Bridge et al., 2025), which is followed by the PE header, containing the information necessary for the programs execution. This includes the number of sections and symbols, and the instruction set used by the program. The section headers contain further metadata about the sections which comprise the program, and reference their location in memory. It also stores a pointer to a table referencing any external functions used by the program, such as the Windows API, and other dynamically linked objects, known as DLLs (Ainapure et al., 2025).

A.7 Word Count

The word-count of this document, as reported by `texcount`, is shown in the following table. Note that this excludes all text found within the appendicies

Chapter	**Word Count**		
	Body Text	Header Text	Outside Text
Introduction	910	17	0
Background	2994	12	52
Methods	2197	16	113
Results	2081	16	215
Discussion	2716	16	215
Total	10898	77	595

Appendix B

Supplimental Results

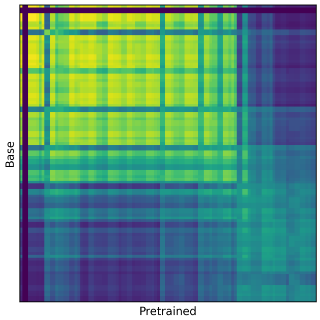
B.1 Obfuscated Benign Model

Classification Metrics

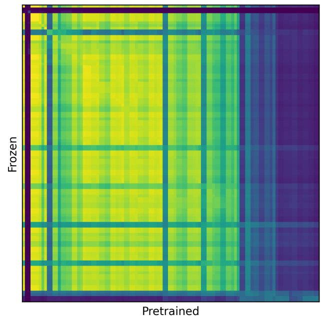
Table B.1: Classification metrics of obfuscated benign sequence classification models.

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8584	0.8587	0.8596	0.8584	0.9638	0.9638	0.9656	0.9637
From Pretrained	0.8704	0.8707	0.8721	0.8704	0.9647	0.9647	0.9667	0.9647
From Pretrained (w/ Frozen Attention Weights)	0.8471	0.8471	0.8472	0.8471	0.9600	0.9600	0.9610	0.9599

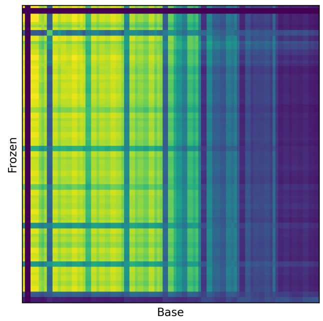
Representational Similarity Analysis



(a) RSA of the pretrained and non-pretrained models.



(b) RSA of the pretrained and frozen models.



(c) RSA of the frozen and non-pretrained models

Figure B.1: Representational Similarity Analysis (RSA) of the obfuscated benign models with Centered Kernel Alignment.

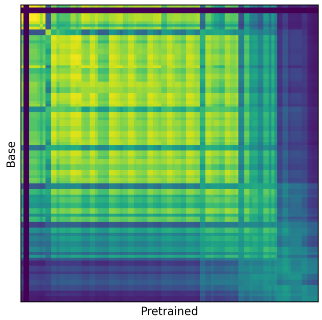
B.2 Partially Obfuscated Benign Model

Classification Metrics

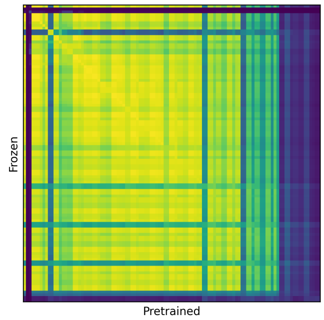
Table B.2: Classification metrics of partially obfuscated benign sequence classification models.

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8282	0.8256	0.8310	0.8282				
From Pretrained	0.8494	0.8481	0.8503	0.8494	0.9152	0.9147	0.9215	0.9140
From Pretrained (w/ Frozen Attention Weights)	0.8053	0.7987	0.8169	0.8053	0.8260	0.8245	0.8335	0.8245

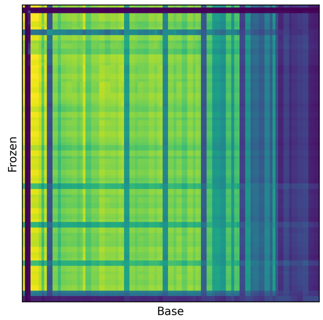
Representational Similarity Analysis



(a) RSA of the pretrained and non-pretrained models.



(b) RSA of the pretrained and frozen models.



(c) RSA of the frozen and non-pretrained models.

Figure B.2: Representational Similarity Analysis (RSA) of the partially obfuscated benign models with Centered Kernel Alignment.

Appendix C

Acknowledgements

This research would not have been possible without the guidance and support of my supervisor, Dr Jeff Mitchell. His expertise and advice has proven to be invaluable, and I am extremely greatful for it.

I would also like to express my gratitude to both Dr Ivor Simpson, and Dr Peter Wijeratne for organising this course. It has been the most challenging, yet deeply rewarding year of my life thus far, and has contributed greatly to my knowledge and professional development.

Many thanks to the lecturers and teaching staff at the University of Sussex whose expertise, and dedication to education have fostered an excellent academic environment. It has been a pleasure to study under all of them throughout my time at this institution.

I'd also like to acknowledge my classmates, whose camaraderie and solidarity have made this journey enjoyable and fulfilling. To Elouan Simonneau, Joe Mills, Jack Speat, and Amy Upson, thank you all for your support, and encouragement. Without you, this year would not have been the same, and I wish you all the best of luck in whatever comes next.

I would also like to thank my housemate Byron, for putting up with me over this past year, and for providing his much needed and appreciated support, boundless kindness, and unwavering understanding.

Last but not least, I would like to express my deepest gratitude to my family for their unwavering support, understanding, and encouragement throughout this academic pursuit. Their love and belief in me have been instrumental throughout my time at the University of Sussex. My sister, Agnes, deserves special thanks for always being available to provide help and support when needed, and for proofreading my work. I wish her the best of luck in her final year.